ARTICLE | Article in Press

DriftDroid: temporal behavioral drift analysis for android malware detection

Muhammet Tan · Halit Bakır · Eid Albalawi · A. H. Abdul Hafez

Received: 26 June 2026 / Accepted: 24 September 2026
© The Author(s) 2026

Abstract

Android malware detectors often perform well under random or temporally mixed evaluation, yet deployed systems classify applications appearing after training. DriftDroid is a temporal behavioral drift measurement protocol for dynamic Android malware detection, not a new detector architecture: it measures how historical runtime-behavior evidence transfers to applications and explains performance decay. KronoDroid supplies 14,348 tabular behavioral profiles and 10,000 event-sequence traces as the historical source domain; a balanced 6,000-sample AndroZoo hold-out from 2024 onward is reserved for testing. Applications are represented with 62 train-domain event tags and 165 aggregate features, evaluated across sequence and tabular probes without target-domain refitting. Although KronoDroid validation scores are high, temporal testing reveals degradation: the best sequence model reaches 0.7988 macro-F1 on AndroZoo, whereas the best tabular model reaches 0.7010. The dominant tabular failure mode is benign-side drift, as recent benign applications increasingly exhibit network, file, cryptographic, SDK, and service behaviors historically correlated with malware. Thus, Android malware evaluation requires temporal hold-outs and representation-aware drift analysis. As secondary operational evidence, tabular tree ensembles recover with 600 labeled future-domain samples, closing 72–80% of the temporal performance gap, whereas sequence models provide stronger unadapted detection, while multi-seed fine-tuning with a separate target-domain validation subset improves both sequence probes and reveals a non-monotonic adaptation response with greater seed/split sensitivity for BiLSTM.

Keywords Android malware · Temporal drift · Dynamic analysis · Behavioral sequence · Concept drift · Realistic evaluation

Introduction

Mobile devices have become indispensable to daily life, with Android commanding approximately 74% of the global mobile operating system market[1]. This dominance, coupled with the platform's open-source architecture, has made Android a primary target for malicious actors. According to Kaspersky's 2023 Mobile Threat Report, over 33.8 million malware, adware, and riskware attacks were blocked on mobile devices, representing a 50% year-over-year increase. More than 1.3 million unique malicious installation packages were identified, of which

This Article in Press is shared early to give you faster access to new research. It is citable and carries a permanent DOI. The final edited version will replace it automatically.



https://doi.org/10.1038/s41598-026-73718-x

Published online: 07 October 2026

scientific reports

154,000 contained mobile banking Trojans. The threat remained active in the following reporting cycle: Kaspersky's 2024 mobile threat analysis reported a 196% increase in Trojan-banker attacks on Android smartphones, from roughly 420,000 attacks in 2023 to 1.24 million in 2024[2]. These escalating figures underscore the urgent need for robust long-term malware detection mechanisms capable of keeping pace with an increasingly sophisticated threat landscape. The central difficulty is not only whether a model can separate malicious and benign applications in a curated benchmark, but whether it remains reliable when the Android ecosystem, malware families, platform APIs, and user-facing application behavior evolve after deployment.

A substantial body of work has addressed Android malware detection through classical machine learning and deep learning. Static detectors use permissions, API calls, intents, manifest declarations, opcodes, or bytecode-derived representations to provide efficient pre-execution screening[3–5]. Dynamic detectors execute applications and monitor runtime behavior such as system calls, API events, and network traffic, thereby capturing behaviors that may be hidden from static inspection[6–9]. Hybrid detectors combine static and dynamic evidence to improve coverage and robustness[10–12]. These lines of work have produced strong reported scores, but many studies still evaluate models using random splits or temporally mixed datasets, allowing future-like samples to influence training and validation. Under such protocols, high accuracy may reflect benchmark convenience rather than deployment readiness.

Recent research has shown that this evaluation gap is not a minor methodological detail. Early empirical studies warned that Android malware evaluation must respect time when measuring generalization[13], and TESSERACT later demonstrated that spatial and temporal bias can substantially inflate malware-classification performance when train/test splits do not match realistic deployment constraints[14,15]. KronoDroid was created specifically to support time-aware Android malware research by providing timestamped hybrid features over a broad historical period[16]. More recent surveys argue that Android malware detection is far from "mission accomplished", identifying outdated datasets, neglected concept drift, dynamic-data consistency, model security, and explainability as unresolved obstacles for long-term detectors[17].

However, existing studies either focus on evaluation protocols or model adaptation, while lacking an explicit analysis of how runtime behaviors evolve over time and how such evolution affects detection performance. This paper therefore shifts the focus from simply proposing another dynamic detector to studying temporal drift in Android runtime behavior. DriftDroid is not a new detector architecture; it is a temporal behavioral drift measurement protocol that uses existing classifiers as controlled probes to quantify and explain how runtime behavior evidence ages. We train on historical KronoDroid samples and reserve recent AndroZoo samples dated from 2024 onward exclusively for testing in the primary source-only temporal evaluation[18]. Each application is executed in a controlled Genymotion-based environment, instrumented at runtime, and represented through both sequential behavioral tags and aggregate dynamic features. The objective is twofold: first, to measure how well behavioral detectors generalize from past to future samples; second, to explain how runtime behavior changes across time. In this framing, model performance is evidence of temporal generalization, while feature distributions, event sequences, and behavior-category prevalence explain the source of degradation.

Research questions

This study is organized around four research questions:

RQ1 Do runtime behavioral detectors trained on historical Android data temporally generalize to applications collected years after training?

RQ2 Which runtime behavior categories and feature distributions shift most from the historical source domain to recent future-domain samples?

RQ3 Which behavioral representation—ordered event sequences or aggregate dynamic profiles—is more temporally stable under benign-side ecosystem evolution?

RQ4 As a secondary operational question, can lightweight adaptation strategies recover temporal performance, and does adaptation efficiency vary structurally across model families?

Contributions

The principal contributions of this work are summarized as follows:

Unlike prior works that treat temporal shift as an evaluation artifact, this study explicitly models and explains behavioral drift in dynamic Android malware detection.

- **Temporal generalization protocol.** We formulate Android malware detection as a temporal generalization problem, use tabular and sequence-based detectors as controlled measurement instruments, and evaluate them under a strict historical-training and future-testing protocol with no future-domain information used during source-model development.
- **Behavior-centric instrumentation pipeline.** We construct a dynamic analysis pipeline that represents runtime activity through 62 interpretable behavioral tags and 165 aggregate dynamic features derived from frequency, temporal, derived, and sequence-chain views.
- **Cross-domain behavioral drift characterization.** We quantify and explain behavioral drift using Jensen–Shannon divergence (JSD), entropy difference, category-level prevalence shifts, and feature-level distributional analysis, identifying which runtime behaviors age and why—in particular, the benign-side SDK and service inflation that collapses historically malware-specific aggregate signals.
- **Secondary adaptation asymmetry study.** As an operational follow-up to the drift analysis, we conduct exploratory adaptation experiments across all seven probes. For sequence probes, fine-tuning is repeated over six seeds using independent target-domain validation and untouched final-test splits. These secondary experiments suggest a data-efficiency asymmetry: tabular tree-ensemble probes close 72–80% of the temporal performance gap with 600 labeled future-domain samples, while sequence probes start from stronger unadapted baselines and improve under fine-tuning, but with non-monotonic BiLSTM sensitivity across seeds.

The remainder of this paper is organized as follows. Section "Related work" reviews related work on Android malware detection, realistic temporal evaluation, concept drift adaptation, and behavioral drift analysis. Section "Materials and methods" details the dynamic analysis and feature extraction methodology. Section "Detection models for temporal evaluation" describes the detection probes. Section "Datasets" presents the dataset construction and cross-domain behavioral drift characterization. Section "Experimental aetup" describes the experimental setup. Section "Results and discussion" reports the results and discussion. Section "Exploratory drift-aware adaptation" presents exploratory adaptation experiments. Section "Threats to validity" discusses threats to validity, and Section "Conclusion" concludes.

Related work

Android malware detection has evolved from static feature classification toward dynamic, hybrid, and sequence-based learning. However, the central question for a deployable detector is no longer only which representation achieves the highest score on a fixed benchmark. A detector must also remain reliable when applications, Android APIs, benign software practices, and malware behavior change over time. This section therefore reviews prior work around four connected themes: Android malware detection paradigms, temporal bias and realistic evaluation, drift adaptation, and behavioral drift interpretation. Across these themes, we emphasize a recurring limitation: much of the literature either improves detection accuracy, constrains evaluation, or adapts models after degradation, but does not explain the behavioral changes that make future samples difficult to classify.

Android malware detection paradigms

Static analysis remains attractive because permissions, manifest entries, API calls, intents, and bytecode features can be extracted without executing an application. Permission- and API-based detectors scale well for pre-installation screening, but their evidence is vulnerable to obfuscation, packing, API evolution, and changing development practices[3–5,19].

Dynamic analysis instead observes runtime behavior such as system calls, API traces, network traffic, and resource interactions[6–9]. Recurrent, attention-based, and transformer models further exploit temporal order by treating API or behavioral logs as sequences[20–24], while hybrid approaches combine static and dynamic evidence[10–12]. These representations improve conventional benchmark discrimination, but they usually treat the dataset as a fixed population and leave open whether dynamic and sequence signals remain stable years after training.

Recent malware-detection studies also emphasize that dataset preprocessing, feature representation, and feature selection materially affect model performance. Work on CLaMP and PE-header-based malware classifiers has used preprocessing pipelines, SVM preprocessing, MRMR ranking, Relief-F feature selection, and feed-forward neural deep features to reduce feature overhead or improve static malware discrimination[25–29]. These works are valuable for efficient static malware detection, whereas the present study asks a complementary question: whether runtime behavioral evidence learned from a historical Android domain remains temporally reliable on recent Android applications.

Broader cybersecurity research further shows that ML-based defense is being extended to 5G-enabled IoT systems, privacy-preserving federated learning, and generative-AI risk assessment[30–32]. These studies motivate the general need for adaptive and privacy-aware security analytics, but they do not replace Android-specific temporal validation because mobile malware detectors face platform, SDK, family, and application-ecosystem drift.

Temporal bias and realistic evaluation

The temporal dimension has long been recognized as a threat to Android malware evaluation. Allix et al[13]. showed that time-aware protocols are necessary because temporally mixed train/test splits can misrepresent deployment generalization. TESSERACT made this concern central by formalizing spatial and temporal constraints and showing that unrealistic splits inflate performance[14]; its extended analysis further confirms that high F1-scores do not settle malware classification when models decay under temporal and spatial shift[15].

Realistic evaluation, however, does not by itself explain failure. A detector evaluated chronologically must classify applications that did not exist during training, and its errors may arise from changing runtime behavior rather than only weaker classification. KronoDroid supports time-aware analysis through timestamped Android features[16], and recent timestamped or self-supervised Android malware studies likewise emphasize temporal verification[33]. The review by Guerra-Manzanares[17] highlights neglected concept drift, outdated datasets, inconsistent

dynamic data, model security, and limited explainability. Thus, prior work establishes the evaluation problem, but not which runtime behaviors shift or how those shifts affect dynamic detectors.

Concept drift handling through online and active learning

Several studies address temporal degradation by updating detectors over time. DroidOL, CASANDRA, AndrO-Det, and DroidEvolver use online, self-evolving, or pseudo-label-based strategies to adapt feature knowledge and classifiers as malware populations change[34–37]. These methods treat drift primarily as a model-maintenance problem: when the distribution changes, the detector must be refreshed.

A related line treats drift as uncertainty management. TRANSCEND and TRANSCENDENT compare new samples with the training distribution or reject uncertain predictions under drift[38,39]. Recent Android-specific work extends this view with active learning, label delay, and streaming retraining[40,41]. These approaches are valuable for deployment, yet they mainly decide when and how to update a model. They do not directly identify which runtime behavior categories changed, how prevalence shifted, or whether sequence-level dynamics themselves drifted.

Representation robustness and behavioral drift analysis

A complementary line asks how to learn features that remain stable across time. APIGraph maps API evidence into semantic relations to reduce model aging[42], TIF targets temporal invariance by discouraging reliance on unstable discriminative features[43], and BYOL-style self-supervised Android malware detection seeks robust representations under timestamp-aware evaluation[33]. Explainability-oriented robust feature selection, such as the LIME/SHAP analysis of Najibi and Bidgoly[44], similarly connects feature robustness with detector reliability, although without a strict historical-train/future-test behavioral drift protocol.

Forensic drift analysis provides the interpretive layer needed here. Zola et al[45]. argue that temporal shifts should be dissected through rolling evaluation, temporal aggregation, misclassification analysis, and feature inspection because performance decay alone does not explain why a model fails. Following this perspective, we compare historical KronoDroid and recent AndroZoo runtime behavior through tag distributions, category frequencies, sequence lengths, entropy, and prevalence changes. The gap addressed here is a dynamic, sequence-aware Android malware study that jointly evaluates future generalization and explains the behavioral drift responsible for it.

Materials and methods

Dynamic analysis infrastructure

Dynamic behavioral analysis of Android applications requires a controlled, reproducible execution environment that can process thousands of APKs at scale while maintaining clean system state between samples. We designed a distributed analysis pipeline built around four parallel Genymotion Android 8.0 emulator instances, each running ARM Translation support to faithfully emulate real-device behavior. The pipeline is composed of three tightly integrated layers: a shell-level orchestration framework, a Python-based analysis coordinator, and a Frida TypeScript instrumentation agent injected into the target process.

Parallel emulator orchestration

The protocol emphasizes controlled, repeatable, and fault-tolerant behavioral acquisition so that downstream learning is based on comparable runtime observations rather than execution artifacts[21,22,24]. We use a parallel

emulator pool composed of independent Android virtual devices; each instance isolates one application at a time, enabling concurrent analysis while preventing cross-sample contamination. Applications are distributed across workers, and periodic emulator resets remove accumulated state such as residual processes, memory fragmentation, and cached artifacts. The orchestration framework monitors emulator health, recovers unresponsive instances, synchronizes APK assignment to avoid duplicate processing, and records checkpoints so interrupted runs can resume without redundant analysis. All samples are executed on the same Android 8.0 image to keep the measurement environment fixed across the historical and recent domains. This design improves longitudinal comparability, but it also means that the reported drift should be interpreted as temporal behavioral transfer under a fixed analysis environment rather than as a clean separation between application-level concept drift and operating-system compatibility effects.

Per-APK analysis protocol

Each APK is analyzed under a deterministic protocol designed for reproducibility, isolation, and behavioral comparability. Before execution, the emulator is restored to a clean baseline; the target application is then installed and launched through its declared activity, falling back to a package-scoped Android Monkey launcher event when no standard entry point is available. Runtime monitoring uses process-level Frida attachment on unmodified APKs, avoiding repackaging and improving compatibility with real-world samples.

Instrumentation is deployed through a reusable precompiled agent, and behavioral data are collected during a fixed 120-second window. After installation, the application receives a 15-second stabilization interval; following attachment, instrumentation state is queried after a one-second warm-up and then polled every five seconds through Frida RPC. To exercise UI- and broadcast-dependent behavior, a package-scoped Monkey process injects up to 500 UI events with an 800 ms throttle, while deterministic stimulation at 15 and 45 seconds sends an SMS-received broadcast, toggles screen state, and posts a test notification. These stimuli are class-independent, increasing behavioral coverage without injecting label-specific signals.

Timeout-aware monitoring and early termination handle non-responsive applications, anti-analysis delays, and instrumentation stalls. At the end of the window, collected behavior is atomically consolidated into the feature representation, the application is removed, and the emulator proceeds to the next sample. The pipeline records behavioral trace density and tag diversity but does not collect instruction-, method-, or basic-block-level code coverage. Consequently, the coverage-related analysis reported below is framed as a runtime-activity proxy for triggered monitored behavior, not as a substitute for full code-coverage instrumentation.

API instrumentation and feature extraction

Frida instrumentation agent

The instrumentation layer is explicitly behavior-centric: it groups runtime events into semantically defined security-relevant categories and preserves their frequency, timing, and ordering simultaneously[22,23]. To capture fine-grained runtime behavior, we design a behavior-centric instrumentation agent based on dynamic API hooking. The agent monitors a diverse set of Android API entry points spanning multiple behavioral categories, including dynamic code loading, network communication, cryptographic operations, file system access, inter-process communication, and user privacy-sensitive actions, as summarized in Table 1.

The instrumentation strategy explicitly models semantically meaningful behavioral groups rather than only coarse aggregate system-call or network summaries. This enables the system to distinguish between functionally similar but contextually different activities, such as benign analytics logging versus privacy-invasive data collection. The “benign SDK” category is a semantic grouping of instrumentation hooks associated with common third-party application services, including analytics and crash-reporting clients, advertising or attribution libraries, social-sharing intents, media playback frameworks, legitimate notification posting, remote configuration,

Table 1 Runtime behavioral taxonomy.

Category	Behavioral examples
Reflection & dynamic loading	Reflection calls, file/memory DEX loading, native libraries
Network & C2 communication	HTTP, sockets, DNS, TLS bypass, remote configuration
Cryptography	AES/RSA use, hashes, Base64, key generation
Surveillance & privacy access	Device identifiers, location, contacts, SMS, microphone
File operations	Sensitive reads, internal/external writes, deletion, bulk encryption
Command execution	Shell and root command execution
Persistence & privilege	Boot receivers, alarms, foreground services, admin requests
Anti-analysis	Debugger, emulator, root, sleep and VM-property checks
Package & IPC behavior	Package enumeration, services, broadcasts, implicit intents
UI and WebView abuse	Clipboard access, JavaScript bridges, WebView execution
Benign SDK signals	Analytics, social sharing, media playback, legitimate notifications

cloud service integration, and authentication flows. The category is not treated as a ground-truth benign label for an application; rather, it records API behaviors that are commonly produced by mainstream SDK functionality and can therefore become confounding aggregate evidence when such behaviors increase in recent benign applications. The resulting runtime vocabulary contains 62 interpretable behavioral tags that form the basis of the sequence modeling and behavioral drift analyses. While several recent transformer-based studies improve dynamic malware analysis through preprocessing, token normalization, or sequence classification, they generally remain focused on distinguishing malicious semantics from raw behavioral content; in contrast, we explicitly model benign SDK signals as a separate category to reduce confusion between legitimate analytics/advertising behavior and privacy-invasive activity with superficially similar API footprints[22,24,46]. Each intercepted API call contributes to three complementary representations. First, global counters track the frequency of behavior across categories. Second, first-occurrence timestamps capture temporal characteristics, enabling the analysis of early-stage execution patterns. Third, all events are recorded in a structured sequential log, preserving ordering information for downstream sequence modeling. To ensure robustness, instrumentation is designed to be non-intrusive and fault-tolerant: failures in individual instrumentation callbacks do not interrupt the analysis process. This guarantees stable data collection even in the presence of incomplete class loading or obfuscated code structures.

Burst behavior modeling

Traditional frequency-based features fail to capture short-lived but high-intensity execution phases commonly observed in malware, such as payload unpacking or rapid data exfiltration. To address this limitation, we introduce a burst-based feature that measures the peak density of API activity within a sliding temporal window. This feature provides a compact representation of execution intensity, enabling the detection of transient but highly indicative behavioral spikes.

Data export

The instrumentation agent outputs a unified behavioral representation combining frequency-based, temporal, and sequential information. This includes aggregated event counts, timing features, burst characteristics, and the ordered API event sequence, forming the foundation for the downstream feature engineering pipeline.

Sequence log throttling

The sequence construction deliberately controls event dominance so that frequent but low-value behaviors do not suppress rare precursor events that are often more diagnostic of multi-stage malicious activity[24,46]. A key challenge in sequence-based dynamic analysis is the imbalance between high-frequency and low-frequency behavioral events. In practice, a small subset of APIs—such as reflection or network communication—can dominate

execution traces, rapidly saturating the sequence log and obscuring rare but security-critical events. Naïve logging strategies that rely on a global event limit tend to over-represent frequent but often benign activities, while under-representing infrequent yet highly discriminative behaviors (e.g., privilege escalation, accessibility abuse, or command execution). This imbalance degrades the effectiveness of downstream sequence models, particularly attention-based architectures that rely on diverse contextual signals. To address this limitation, we introduce a category-aware throttling mechanism that enforces per-event-type constraints during sequence construction. Rather than allowing dominant behaviors to monopolize the log, this approach ensures that multiple behavioral categories are proportionally represented within each execution trace. Importantly, this design preserves rare but diagnostically significant events while preventing noise saturation caused by repetitive API calls. As a result, the resulting sequence representation maintains both diversity and discriminative power, improving the model's ability to capture meaningful behavioral patterns across applications. This design is particularly important for capturing multi-stage attack patterns, where rare precursor events may otherwise be lost in dense execution traces.

Feature engineering pipeline

The feature engineering pipeline integrates four complementary views of runtime behavior: direct event intensity, higher-order derived semantics, temporal execution dynamics, and explicit sequence-chain abstractions[22–24]. Raw outputs produced by the instrumentation agent are transformed into a structured tabular representation through a multi-stage feature engineering pipeline. The exported tabular profile contains 165 numeric behavioral features and three metadata fields, organized into raw counters, derived features, temporal features, and sequence-based features (Table 2). This design captures not only the presence of behavioral signals, but also their intensity, temporal characteristics, and contextual relationships.

Raw behavioral counters

The first group consists of direct API call frequencies aggregated across all monitored behavioral categories (Table 2). These features provide a coarse-grained view of application activity, capturing the overall intensity of operations such as dynamic code loading, cryptographic usage, network communication, file system interaction, and privacy-sensitive actions. While informative, raw counts alone are insufficient for distinguishing benign and malicious applications due to their sensitivity to execution duration and application complexity. Therefore, they serve as a foundational layer upon which more discriminative features are constructed.

Derived behavioral features

To overcome the limitations of raw frequency signals, we construct a set of derived features that capture higher-level behavioral semantics (Table 2). These include composite behavioral scores, inter-category ratios, Boolean pattern indicators, and normalized activity measures. Composite scores aggregate related API calls into interpretable behavioral dimensions such as network activity, persistence, stealth, and exfiltration. To mitigate bias introduced by varying execution lengths, selected features are normalized by session duration, enabling fair comparison across applications. Behavioral ratios further capture relationships between categories, providing invariance to absolute activity levels. In addition, Boolean pattern flags encode compound behavioral signatures that are difficult to capture through individual features alone, such as the co-occurrence of encoding and network transmission or the combination of privilege escalation and dynamic execution. This multi-view design is motivated by the limitations of raw-count and sequence-only modeling: frequency-based approaches ignore temporal context, whereas pure sequence models underutilize stable aggregate behavioral evidence unless it is explicitly encoded[22,47]. Finally, rate-based and log-transformed features are introduced to stabilize high-variance distributions and improve robustness for gradient-based learning models.

Table 2 Extracted behavioral feature schema.

Group	Sub-group	Count
Raw Counters	Reflection & dynamic loading	5
Raw Counters	Cryptography	5
Raw Counters	Anti-analysis	7
Raw Counters	General behavior	2
Raw Counters	Intent & IPC	6
Raw Counters	File system	6
Raw Counters	Network & socket	5
Raw Counters	Persistence & privilege	5
Raw Counters	Process & memory	5
Raw Counters	Accessibility, overlay & clipboard	4
Raw Counters	Telephony & device identity	4
Raw Counters	Extended v4 counters	25
Raw Counters	**Sub-total**	**79**
Derived Features	Composite behavioral scores	8
Derived Features	Behavioral ratios	5
Derived Features	Boolean pattern flags	7
Derived Features	Session-normalized scores	4
Derived Features	Per-second rate features	6
Derived Features	Log1p outlier-stable features	6
Derived Features	**Sub-total**	**36**
Temporal Features	First-occurrence timestamps	8
Temporal Features	Burst & session duration	2
Temporal Features	Temporal ordering flags	4
Temporal Features	Early-5s activity flags	3
Temporal Features	Inter-event deltas	4
Temporal Features	Session anomaly flag	1
Temporal Features	**Sub-total**	**22**
Sequence Features	Binary chain presence flags	8
Sequence Features	Chain statistics	2
Sequence Features	Chain occurrence counts	5
Sequence Features	First chain completion times	5
Sequence Features	Attack vector chain flags & counts	4
Sequence Features	Attack vector chain timings	4
Sequence Features	**Sub-total**	**28**
Behavioral feature total		**165**
Metadata fields	Package name, label, timestamp	**3**
Exported columns		**168**

Temporal features

The third group captures the temporal dynamics of application behavior (Table 2). These features are derived from the timing of first-occurrence events and encode when specific behaviors emerge during execution. Early-stage activity is particularly informative in malware detection, as malicious applications often trigger payload execution shortly after launch. To capture this phenomenon, we include early-activity indicators, inter-event timing relationships, and ordering constraints between behavioral categories. These timing-based features are highly discriminative within the source domain; however, as the feature-group ablation reveals, their absolute timing patterns can overfit to the historical behavioral cadence and reduce temporal transfer performance when the target domain exhibits different API-invocation timing—a subtlety that is not visible without an explicit temporal evaluation protocol. Additionally, session-level indicators such as burst intensity and execution duration are incorporated to detect anomalous runtime patterns, including stalling behavior or delayed execution used to evade sandbox analysis.

Sequence-based behavioral features

The final group extracts higher-order behavioral patterns from the sequential API event log (Table 2). Rather than relying solely on token sequences for downstream models, we explicitly encode structured interaction patterns between events. These features capture ordered co-occurrence relationships, such as data access followed by network transmission or persistence setup preceding execution. Such patterns are particularly indicative of multi-stage attack workflows and cannot be effectively represented through isolated counters or timestamps. We further incorporate chain-level statistics, including occurrence counts and timing of first appearance, enabling the model to reason about both the presence and progression of complex behavioral sequences.

Dual-format output

The pipeline generates two complementary output formats per analysis run to preserve both structured summaries and fine-grained behavioral dynamics.

Tabular behavioral profiles

The extracted tabular features provide a fixed-length behavioral profile for each analyzed APK, supporting gradient-boosted tree evaluation and other lightweight tabular baselines. Each APK is represented by 168 columns, consisting of 165 numeric behavioral features and three metadata fields. This representation captures stable aggregate, temporal, and derived behavioral characteristics regardless of execution length.

Event-sequence traces

In parallel, a sequence-based representation is constructed, preserving the ordered runtime event trace along with session metadata. This representation supports sequence-based models, particularly transformer architectures that rely on contextual relationships between events. The current sequence pipeline retains the available event traces and pads or truncates event streams dynamically during batching, using a maximum sequence length of 256 events. Very short execution traces are treated as low-context observations during modeling rather than removed from the tabular feature space, since aggressive sequence filtering can bias the dataset toward highly active applications. By preserving both tabular and sequential views, the pipeline enables complementary modeling of global behavioral patterns and temporal dependencies.

Detection models for temporal evaluation

The central contribution of this work is the temporal and behavioral drift analysis, not a new classifier architecture. We therefore use representative sequence and tabular classifiers as measurement instruments for evaluating how different behavioral representations generalize from historical to future applications. All model selection, vocabulary construction, preprocessing, scaling, and threshold choices are performed only within the KronoDroid source domain before applying the trained models to the AndroZoo temporal hold-out.

Sequence-based probes

The sequence experiments use the ordered runtime event trace as input. Each event is mapped to a train-only behavioral vocabulary of 62 tags (Table 3), and traces are padded or truncated to a maximum length of 256 events. Two sequence probes are evaluated. The BiLSTM captures bidirectional local and medium-range temporal dependencies in event order, while the Transformer encoder captures longer-range contextual interactions

through self-attention. Both models use the same KronoDroid train/validation split and the same frozen vocabulary at temporal test time.

Tabular behavioral probes

The tabular experiments use the 165-dimensional behavioral feature vector after excluding package name, label, and timestamp metadata. The tabular probes cover both classical and neural model families: logistic regression as a linear reference, random forests, extremely randomized trees, and XGBoost as tree-based baselines for heterogeneous behavioral counts and ratios, and an MLP as a neural tabular baseline. These models are included to test whether aggregate behavioral profiles are more or less temporally stable than ordered sequence traces.

Evaluation models

Table 4 summarizes the evaluation models. The comparison is intentionally representation-oriented: sequence models test temporal ordering, while tabular models test aggregate behavioral profiles. A fused multimodal architecture is outside the scope of this study; the objective is to characterize temporal drift across representation types, not to optimize a combined detector.

Datasets

The study is positioned as a temporal generalization and behavioral drift analysis rather than an online-learning or continual-adaptation benchmark. KronoDroid is used as the historical source domain, while recent AndroZoo applications are reserved as a future-domain temporal hold-out. The completed AndroZoo target set is used as a balanced 6,000-application temporal hold-out dated from 2024 onward, with 3,000 malware and 3,000 benign samples (Table 5). Figure 1 summarizes the completed KronoDroid source-domain composition: 14,348 dynamically analyzed tabular profiles and 10,000 event-sequence traces with valid runtime traces. An additional 147 benign tabular profiles are retained as reserve samples and are not included in the balanced temporal evaluation.

The completed KronoDroid data are split only within the historical domain for model development. For tabular experiments, 12,195 applications are used for training and 2,153 for validation, preserving the near-balanced label ratio. For sequence experiments, the balanced 10,000-application trace subset is split into 8,500 training and 1,500 validation applications. Vocabulary construction, feature scaling, model selection, and thresholding are all fit only on the KronoDroid training split. This prevents future-domain information from influencing model selection before AndroZoo evaluation.

KronoDroid provides temporally diverse historical samples spanning Android ecosystem evolution from 2008 to 2020[16], whereas the AndroZoo split introduces more recent benign and malicious applications that better reflect contemporary SDK usage patterns and current malware behaviors. Candidate AndroZoo samples are selected using collection-date metadata and VirusTotal consensus. For the experiments reported here, the candidate list and VirusTotal consensus metadata were frozen in May 2026 before feature extraction. Malware candidates are restricted to applications dated 2024 or later and flagged as malicious by at least three VirusTotal engines. Benign candidates are restricted to applications dated 2024 or later with zero VirusTotal malicious detections. Samples with low-positive or otherwise ambiguous VirusTotal evidence are excluded from the temporal hold-out to reduce label noise. For the secondary adaptation analysis, the frozen AndroZoo target set is partitioned into disjoint adaptation, validation, and final-evaluation subsets, as detailed in Section Exploratory drift-aware adaptation.

Verified malware-family labels are not used as model inputs and are not assumed to be complete for the temporal hold-out. This is deliberate: the study evaluates behavioral transfer under a clean source-versus-future split rather than family-level attribution. When coarse family or application-category metadata are available, they are used only as a sanity check to avoid an obviously single-family or single-category target set; samples without

Table 3 KronoDroid behavioral event-tag vocabulary.

Tag	Behavioral semantics	Tag	Behavioral semantics
Anti-Analysis & Evasion			
ANTI_DEBUG	Debugger attachment detection/ evasion	ANTI_EMULATOR	Emulator/sandbox environment detected
ANTI_ROOT_CHECK	Rooted-device check	ANTI_SLEEP_LONG	Long sleep to evade analysis timeout
ANTI_VM_PROPS	Reads VM-specific device properties		
Code Loading & Shell Execution			
DEX_LOAD_FILE	DEX/JAR bytecode loaded from file	DEX_LOAD_MEMORY	DEX/JAR loaded from memory buffer
EXEC_ROOT_CMD	Shell command executed with root privilege	EXEC_SHELL_CMD	Arbitrary shell command executed
NATIVE_LIB_LOAD	Native (.so) library dynamically loaded		
Cryptography			
CRYPTO_AES	AES symmetric encryption invoked	CRYPTO_BASE64_DEC	Base64 decoding
CRYPTO_BASE64_ENC	Base64 encoding	CRYPTO_HASH_MD5	MD5 hash computation
CRYPTO_HASH_SHA	SHA-family hash computation	CRYPTO_KEYGEN	Cryptographic key generation
CRYPTO_RSA	RSA asymmetric encryption invoked		
File Operations			
FILE_DELETE	File deleted	FILE_ENCRYPT_BULK	Bulk file encryption (ransomware pattern)
FILE_READ_SENSITIVE	Sensitive path or file read	FILE_WRITE_EXTERNAL	File written to external storage
FILE_WRITE_INTERNAL	File written to internal storage		
Network & C2			
NET_DNS_LOOKUP	DNS name resolution	NET_HTTP_GET	HTTP GET request sent
NET_HTTP_POST	HTTP POST request sent	NET_SOCKET_TCP	Raw TCP socket connection established
NET_SOCKET_UDP	UDP socket communication	NET_SSL_BYPASS	SSL certificate validation bypassed
IPC, Package & Persistence			
IPC_BIND_SERVICE	Binds to background service	IPC_IMPLICIT_INTENT	Implicit broadcast intent sent
IPC_SEND_BROADCAST	Explicit broadcast message sent	PKG_DISABLE	Installed package disabled
PKG_ENUMERATE	Installed packages enumerated	PERSIST_ALARM_REP	Repeating alarm registered
PERSIST_BOOT_RECV	Boot-completed broadcast receiver set	PERSIST_FG_SERVICE	Foreground service started
PERSIST_JOB_SCHED	Background job scheduled	PERM_ADMIN_REQ	Device administrator privilege requested
Reflection			
REFLECT_CLASS_LOAD	Class loaded via Java reflection	REFLECT_FIELD_ACCESS	Field accessed via reflection
REFLECT_INVOKE	Method invoked via reflection		
Surveillance & Privacy			
CLIPBOARD_ACCESS	Clipboard content read	KEYLOG	Keystroke logging detected
KEYSTORE_EXFIL	Android Keystore accessed for exfiltration	PHONE_CALL_SILENT	Silent phone call initiated
PHONE_STATE_LISTEN	Phone state changes monitored	SURV_CALL_LOG	Call log read
SURV_CONTACT_READ	Device contact list read	SURV_DEVICE_ID	Device identifier (IMEI/ MEID) read
SURV_LOCATION_GPS	GPS location accessed	SURV_LOCATION_NET	Network-based location accessed
SURV_MIC_RECORD	Microphone audio recorded	SURV_SMS_READ	SMS messages read
UI, Media & WebView			
MEDIA_PHOTO_ACCESS	Camera or photo gallery accessed	MEDIA_PLAY	Media file playback invoked
MEDIA_VIDEO_ACCESS	Video content accessed	NOTIF_POST_LEGIT	User-visible notification posted
SOCIAL_SHARE	Social sharing intent invoked	WEBVIEW_JS_BRIDGE	JavaScript-to-Java bridge registered
WEBVIEW_JS_EXEC	JavaScript executed inside WebView		

Table 3 (continued)

Tag	Behavioral semantics	Tag	Behavioral semantics
Analytics & Remote Config			
ANALYTICS_LOG	Analytics event logged	REMOTE_CONFIG_FETCH	Remote configuration fetched

Table 4 Evaluation models used as temporal generalization probes.

Representation	Model	Purpose
Sequence	BiLSTM	Recurrent sequence-order baseline
Sequence	Transformer	Attention-based sequence baseline
Tabular	Logistic Regression	Linear aggregate-behavior reference
Tabular	Random Forest	Classical tree ensemble baseline
Tabular	Extra Trees	Randomized tree ensemble baseline
Tabular	XGBoost	Gradient-boosted tabular baseline
Tabular	MLP	Neural aggregate-feature baseline

Table 5 Dataset composition used in the current temporal evaluation.

Source/representation	Benign	Malware	Total
KronoDroid tabular profiles	7,197	7,151	14,348
KronoDroid event-sequence traces	5,000	5,000	10,000
AndroZoo 2024–2026 temporal hold-out	3,000	3,000	6,000

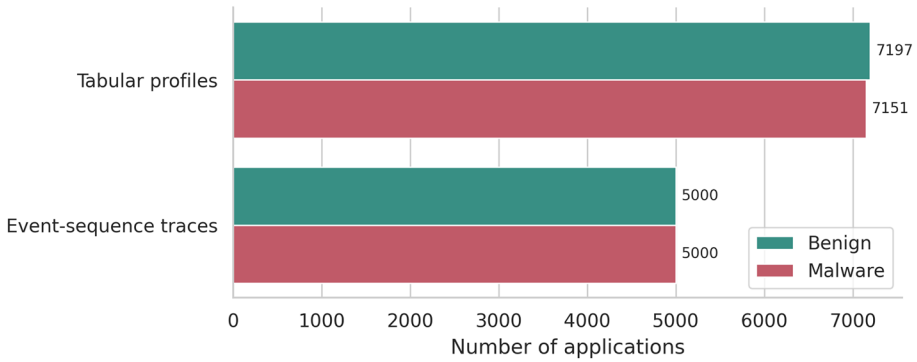


Fig. 1 KronoDroid source-domain composition.

reliable family/category metadata remain in the evaluation because excluding them would bias the temporal hold-out toward easier-to-label applications. Because family labels are incomplete and often derived from noisy antivirus naming conventions, we do not infer malware families post hoc from package names or vendor labels. Instead, the main stratified analysis is performed at the monitored behavior-category level, where labels are generated by the instrumentation taxonomy itself and are consistently available for every valid trace.

To prevent data leakage, we perform cross-source deduplication before feature extraction and model training. In particular, APKs collected from KronoDroid and AndroZoo are compared using their SHA-256 hashes, and duplicate samples are removed prior to split construction. We additionally verify package identifiers to detect repeated applications with different collection metadata. This procedure ensures that no identical APK appears in both the training/validation pool and the held-out temporal evaluation set. A lightweight Python-based preprocessing script is used to enforce this uniqueness constraint and to generate the final leakage-free sample lists.

All APKs undergo dynamic analysis as described in the per-APK analysis protocol and API instrumentation sections. Samples that fail to install, fail to produce a valid process identifier within the retry window, or trigger repeated instrumentation timeouts are recorded in the failure log and excluded from the final dataset. Deduplication and exclusion are completed before model fitting to ensure that downstream evaluation reflects only valid and unique applications.

This setup enables a realistic evaluation scenario in which the model must generalize across temporal distribution shifts, rather than relying on randomly shuffled datasets that may overestimate real-world performance. The tabular pipeline represents each successfully analyzed application using a 168-column behavioral profile, including 165 numeric dynamic features and three metadata fields. The sequence pipeline uses a 62-tag active KronoDroid vocabulary and a maximum sequence length of 256 events, with unseen AndroZoo tags mapped to an unknown token during temporal inference. This dual-format design supports both aggregate behavioral modeling and sequence-level drift analysis.

Source-domain statistical characterization

Before evaluating temporal drift against AndroZoo, we characterize the completed KronoDroid source domain to establish the behavioral quantities that will later be compared across time. This characterization is not presented as the main drift result; rather, it defines the measurement layer for the future historical-versus-recent comparison. Event-trace length already shows strong class-conditional structure: benign applications contain a median of 18 events per trace (mean 31.6, 95th percentile 107), whereas malware contains a median of 50 events (mean 61.6, 95th percentile 168) (Fig. 2). A Mann–Whitney U test and a two-sample Kolmogorov–Smirnov test both reject equality of the length distributions ($p \ll 0.001$, Cliff's $\delta = 0.568$), indicating a large shift toward longer and denser malware traces.

At the behavioral-category level, malware exhibits higher application-level prevalence for surveillance and privacy access (80.6% versus 17.0%), file operations (78.3% versus 39.8%), cryptography (71.7% versus 37.4%), network and C2 behavior (85.3% versus 54.0%), and persistence or privilege-related behavior (44.0% versus 19.3%) (Fig. 3). These differences are statistically significant under chi-square tests after category aggregation. The same prevalence estimates are then computed for the recent AndroZoo hold-out and compared using effect sizes and multiple-testing-aware significance analysis, allowing the paper to explain not only whether performance changes, but which runtime behaviors changed.

The tabular behavioral profiles provide a complementary view of this separation (Fig. 4). Device-identifier access is observed in 57.2% of malware but only 3.4% of benign applications, while surveillance, network, exfiltration, stealth, and file-write/read behaviors also show large prevalence gaps. These source-domain differences are useful for detector training, but they are also exactly the type of evidence that can become unstable over time. The drift evaluation will therefore report feature-prevalence changes, Jensen–Shannon divergence, entropy difference, and category-level effect sizes between KronoDroid and the completed AndroZoo hold-out.

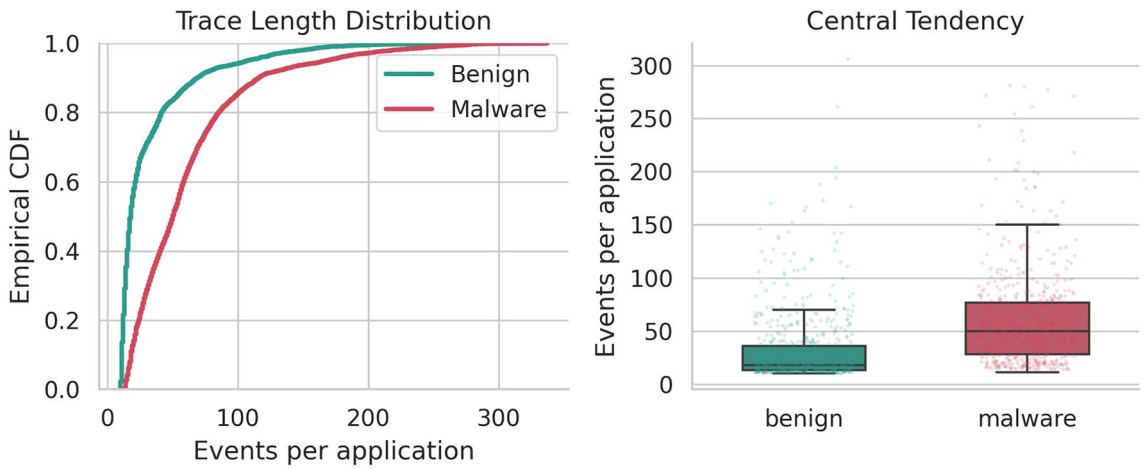


Fig. 2 KronoDroid event-trace length distribution.

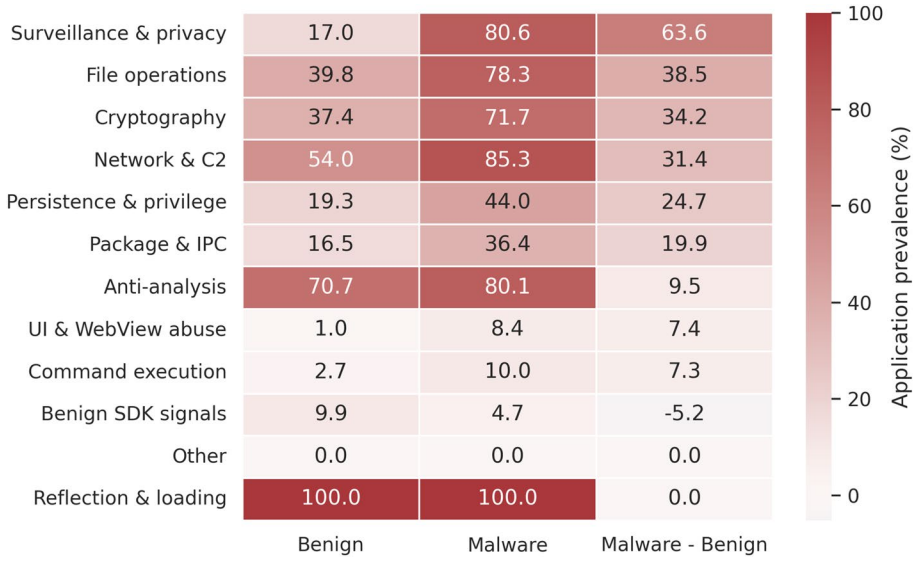


Fig. 3 Runtime behavior-category prevalence in KronoDroid.

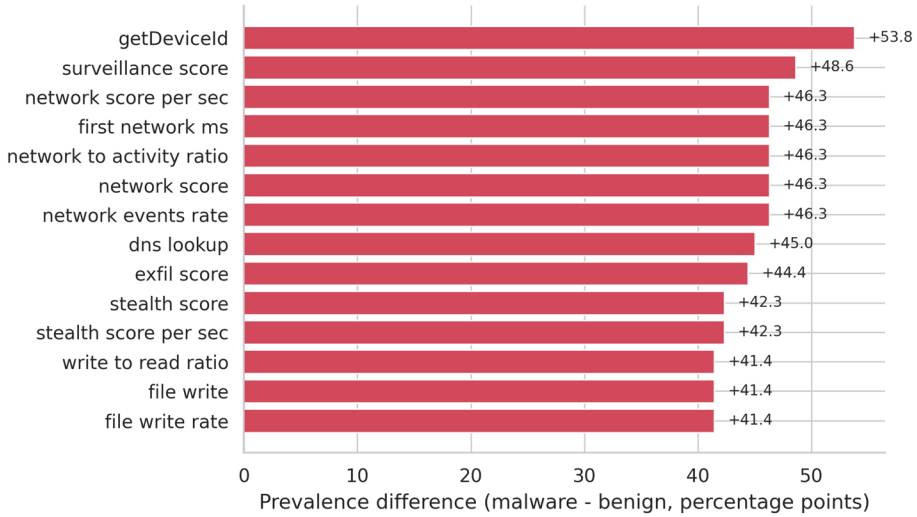


Fig. 4 Most class-separated dynamic tabular behaviors in KronoDroid.

Cross-domain behavioral drift characterization

The completed AndroZoo hold-out enables a direct comparison between the historical KronoDroid source domain and recent applications collected from 2024 onward. The most visible shift appears in runtime trace density. In KronoDroid, malware traces are substantially longer than benign traces, with medians of 50 and 18 events, respectively. In the recent AndroZoo hold-out, this class-conditional gap nearly disappears: benign and malware traces have median lengths of 96 and 95.5 events, respectively (Fig. 5). This indicates that trace length alone is not a stable malware indicator across time. Recent benign applications generate denser runtime behavior, likely due to contemporary SDKs, authentication libraries, cloud services, telemetry, file-cache operations, and push or notification mechanisms.

To further assess whether monitored behavior is actually triggered during the fixed execution window, Table 6 reports runtime-activity proxy statistics computed from the event-sequence logs. These statistics do not measure code coverage; rather, they summarize how many monitored events and behavioral tags were observed within the 120-second analysis window and during the first 30 seconds of execution.

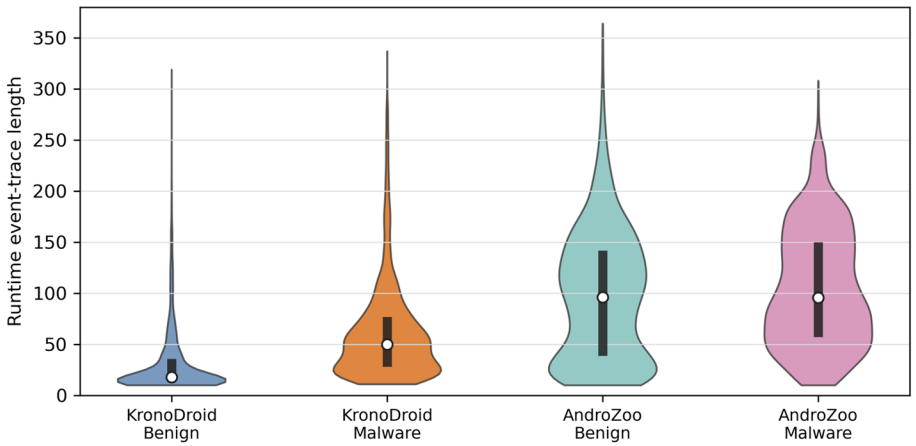


Fig. 5 Cross-domain runtime event-trace length comparison.

Table 6 Runtime-activity proxy statistics for the 120-second analysis window. These values summarize triggered monitored behavior, not instruction- or method-level code coverage.

Subset	N	Seq. len. median	Seq. len. mean	Seq. len. 95th pct.	Active tags	Unique tags/app median	Events ≤ 30s median	Duration median (s)
row KronoDroid benign	5,000	18.0	31.6	107	59	6.0	14.0	121.8
row KronoDroid malware	5,000	50.0	61.6	168	58	11.0	43.0	121.6
row AndroZoo benign	3,000	96.0	97.9	208	68	14.0	84.0	121.7
row AndroZoo malware	3,000	95.5	104.4	202	63	23.0	90.0	84.1

The proxy statistics show that the recent AndroZoo traces are not systematically less stimulated than the historical traces. In fact, recent benign and malware samples produce denser monitored traces and more unique tags per application than their KronoDroid counterparts, supporting the interpretation that the observed degradation is not simply caused by an absence of triggered behavior in the target domain. The lower median observed duration for AndroZoo malware (84.1 s) suggests that many recent malware traces emit monitored events more quickly, and may also reflect earlier termination, crash behavior, or anti-analysis responses in a subset of samples. At the same time, because these are behavioral-log statistics rather than code-coverage measurements, they cannot exclude unobserved dormant payloads or API paths outside the monitored taxonomy.

Vocabulary-level drift provides further evidence that the future-domain samples are not merely a resampled version of the source domain. KronoDroid contains 62 active behavioral tags, whereas the recent AndroZoo traces contain 69 active tags, with 60 tags shared across both domains. New AndroZoo-only tags include advertising SDK initialization, OAuth authentication, cloud storage, crash reporting, map APIs, push-channel registration, WebSocket communication, hook detection, and root-hiding behavior. These additions reflect both benign ecosystem evolution and contemporary evasion or communication patterns, reinforcing the need to evaluate sequence models under a strict temporal split.

Figure 6 shows that category-level prevalence drift affects benign and malicious applications differently. Benign applications show large increases in file operations (+54.1 percentage points), cryptography (+49.4), persistence or privilege behavior (+43.9), package and IPC behavior (+35.6), and network/C2 activity (+22.8). Malware also shifts, but in a different direction: command execution increases by +6.4 percentage points, package and IPC behavior by +19.9, UI, WebView, and SDK activity by +24.7, persistence or privilege behavior by +21.2, and cryptography by +22.7, while surveillance and privacy-access prevalence decreases by −21.0. This asymmetric drift suggests that historical malware discriminative behaviors can become less class-specific as benign applications adopt richer SDK and service interactions.

The tabular profiles confirm the same pattern at feature level (Fig. 7). The largest benign-domain increases are associated with file deletion and writing, sensitive file reads, dynamic class loading, and early file-write behavior.

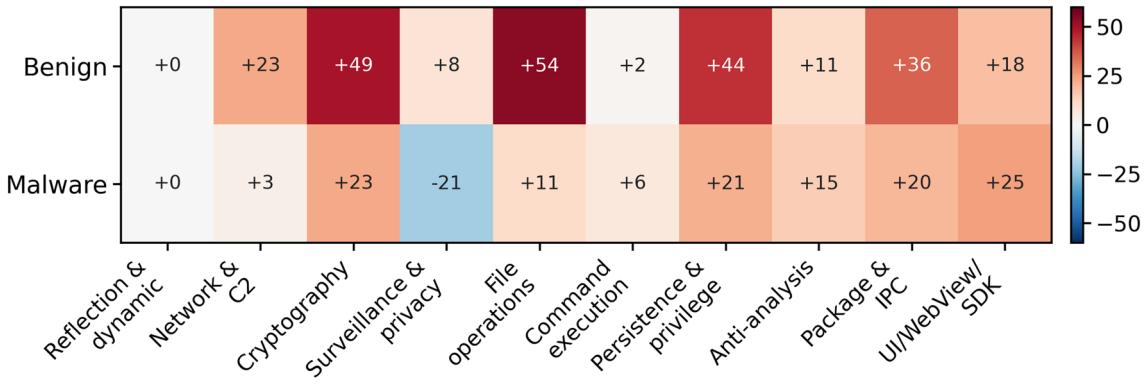


Fig. 6 Category-level prevalence drift from KronoDroid to AndroZoo.

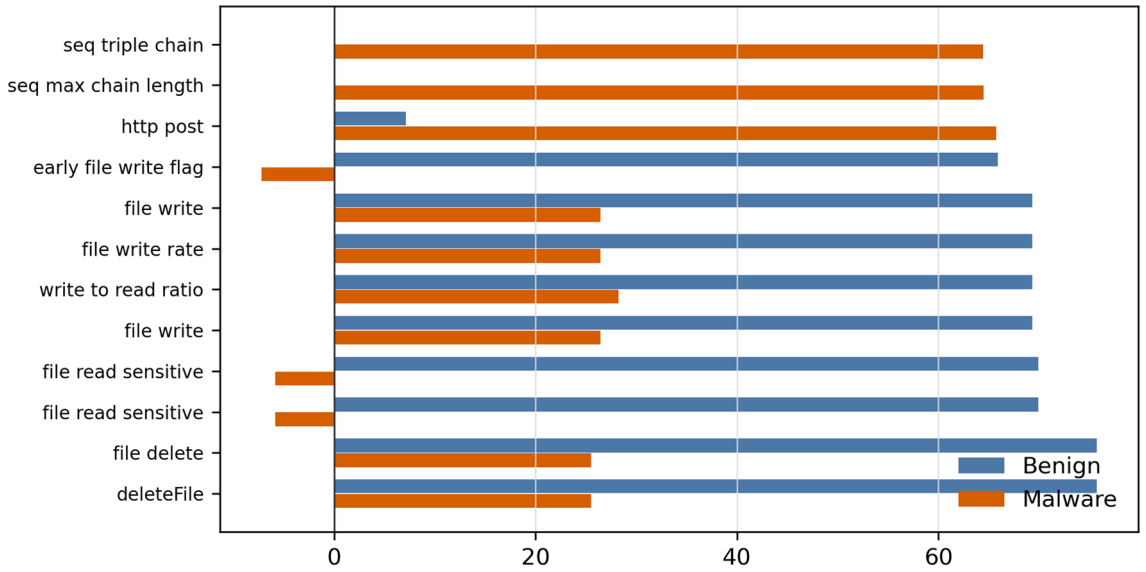


Fig. 7 Largest dynamic-feature prevalence changes across domains.

In contrast, malware-domain increases are concentrated around HTTP POST activity, sequence-chain statistics, crypto-network chains, and anti-analysis or payload-ordering patterns. These observations refine the paper's central claim: temporal degradation is not only a matter of lower accuracy on future samples, but a measurable shift in which runtime behaviors remain discriminative.

Experimental setup

The experimental protocol uses KronoDroid for model development and reserves AndroZoo for temporal out-of-distribution testing. Within KronoDroid, training and validation are stratified, while all preprocessing decisions that can leak distributional information are fit only on the training split. For tabular experiments, metadata fields are excluded from model input, yielding 165 behavioral dimensions. The tabular evaluation uses representative classical and neural probes, including logistic regression, random forests, extremely randomized trees, XGBoost, and an MLP. Each complete preprocessing-and-model pipeline is fit only on KronoDroid and then applied unchanged to the balanced AndroZoo temporal hold-out.

For the completed sequence experiment, each event trace is encoded using a train-only vocabulary and padded or truncated to 256 events. The active KronoDroid training vocabulary contains 62 behavioral tags plus PAD and UNK tokens. We evaluate two representative sequence probes: a bidirectional LSTM and a Transformer encoder. Both models are trained only on the KronoDroid training split, selected using KronoDroid validation performance, and then applied once to the balanced AndroZoo temporal hold-out without refitting vocabulary, thresholds, or model parameters.

Reproducibility details

The primary source-only temporal evaluation uses a fixed random seed of 42 and a stratified 85/15 KronoDroid train/validation split. In that evaluation, the frozen AndroZoo set is used only once for temporal inference and drift characterization; it is not used for preprocessing, source-domain model selection, vocabulary construction, threshold tuning, or early stopping. The separate sequence-adaptation analysis re-partitions the same frozen AndroZoo set over six random seeds (11, 22, 33, 42, 44, and 55) into disjoint adaptation, validation, and final-test subsets, and fine-tunes only the sequence probes under that secondary protocol. Binary decisions in the source-only temporal evaluation use a fixed 0.5 malware-probability threshold, so the reported false-positive and false-negative rates reflect deployment-style transfer rather than target-domain optimization.

Table 7 summarizes the main experimental settings. Sequence traces are capped at 256 events because this length preserves the majority of source-domain traces, including the 95th percentile of KronoDroid malware traces, while bounding memory and training time. Unseen AndroZoo event tags are mapped to an unknown token, preserving the strict train-past/test-future protocol. For tabular models, median imputation is fit on the KronoDroid training split; robust scaling is used only for scale-sensitive logistic regression and MLP probes, while tree-based models use unscaled imputed features. Hyperparameters are intentionally standard and modest rather than exhaustively tuned, because the models are used as representation probes for drift evaluation rather than as a new detector architecture. The table also documents the settings most likely to affect replication: the fixed threshold, train-only vocabulary, neural optimization parameters, tabular preprocessing, and tree-ensemble sizes. Together with the dataset composition in Table 5, these settings define the complete experimental protocol used to produce the results in Table 8.

Table 7 Key reproducibility settings for the temporal evaluation.

Component	Setting	Rationale
Split protocol	Stratified 85/15 KronoDroid validation; AndroZoo held out	Prevents future-domain leakage during model development
Decision threshold	0.5 for all models and domains	Avoids target-domain threshold tuning
Sequence encoding	Train-only vocabulary; PAD/UNK tokens; maximum length 256	Preserves temporal order while handling unseen future tags
Sequence optimization	AdamW, learning rate 10^{-3}, weight decay 10^{-4}, batch size 128	Stable baseline training for neural sequence probes
Sequence training	Up to 20 epochs, early stopping patience 5, validation ROC-AUC selection	Selects models only from source-domain validation evidence
BiLSTM	Embedding 96, hidden size 128, 2 layers, dropout 0.2	Captures bidirectional local and medium-range event dependencies
Transformer	Embedding 96, 4 heads, 2 encoder layers, feed-forward size 256, dropout 0.2	Captures longer-range event interactions with a compact encoder
Tabular preprocessing	Median imputation; robust scaling for logistic regression and MLP	Handles missing values and count-scale heterogeneity without target leakage
Tree ensembles	Random Forest/Extra Trees with 500 trees; XGBoost with 600 trees, depth 5, learning rate 0.04	Provides strong aggregate-feature baselines without architecture novelty claims
MLP	Hidden layers 256–128–64, ReLU, batch size 256, early stopping	Neural tabular baseline for aggregate dynamic features

Artifact availability

The experimental notebooks are organized under the project experiment directory and separately cover KronoDroid sequence training, KronoDroid tabular training, AndroZoo sequence inference, and AndroZoo tabular inference. The tabular notebooks export complete preprocessing-and-model pipelines, while the sequence notebooks export the trained BiLSTM and Transformer checkpoints together with the frozen vocabulary and metadata required for temporal inference. The feature-extraction implementation is maintained as a separate dynamic-analysis artifact. Dataset acquisition follows the standard access paths for KronoDroid and AndroZoo; the released artifacts focus on the study-specific protocol, feature schemas, trained-model metadata, derived behavioral feature tables, and split reconstruction material where permitted.

Results and discussion

The central empirical message is that temporal degradation is primarily a benign-side behavioral convergence problem, not simply malware evolution. Table 8 reports the completed source-domain validation and temporal hold-out results for both representation families. The high KronoDroid validation scores show that the probes can learn discriminative historical runtime patterns. However, every model degrades on the recent AndroZoo hold-out, confirming that source-domain validation alone overestimates future-domain reliability.

The temporal degradation is not uniform across model families. The Transformer obtains the strongest KronoDroid validation ROC-AUC (0.9905) and remains slightly better ranked by AndroZoo ROC-AUC (0.8525), but its fixed-threshold malware recall degrades sharply on the temporal hold-out, producing a false-negative rate of 30.2%. The BiLSTM has a lower AndroZoo ROC-AUC (0.8371), yet it provides a more balanced fixed-threshold decision profile, with higher AndroZoo macro-F1 (0.7988 versus 0.7746) and a substantially lower false-negative rate (19.4%). This change in model ordering under temporal evaluation reinforces the paper's central point: source-domain validation is insufficient for selecting a detector intended for future samples.

The tabular probes show an even stronger representation-level shift. On KronoDroid validation, the strongest tree-based models are competitive with the sequence probes: XGBoost, Random Forest, and Extra Trees all reach macro-F1 values around 0.953–0.954 and ROC-AUC values above 0.977. On the AndroZoo hold-out, however, the best tabular macro-F1 is 0.7010 for XGBoost, and the remaining tabular probes fall between 0.5869 and 0.6446. The main failure mode is not an inability to detect malware; Random Forest, Extra Trees, and MLP keep false-negative rates near 10%. Instead, they classify many recent benign applications as malware, with false-positive rates between 57.4% and 62.2%. This pattern is consistent with the dataset-level drift analysis: modern benign applications contain more network, file, cryptographic, SDK, and service behaviors that historically served as malware-correlated aggregate signals.

Because operational malware screening often requires much lower false-positive rates than a balanced measurement benchmark, we also inspect strict low-FPR operating points using the frozen AndroZoo prediction

Table 8 Source-domain validation and temporal hold-out results.

Rep.	Model	Krono F1	Krono AUC	AZ F1	AZ AUC	Δ F1	AZ FPR	AZ FNR	AZ PR-AUC
Sequence	BiLSTM	0.9587	0.9885	0.7988	0.8371	−0.1598	0.2080	0.1943	0.7968
Sequence	Transformer	0.9627	0.9905	0.7746	0.8525	−0.1880	0.1460	0.3020	0.8591
Tabular	XGBoost	0.9544	0.9835	0.7010	0.7399	−0.2535	0.4403	0.1447	0.7060
Tabular	Random Forest	0.9544	0.9806	0.6446	0.7409	−0.3098	0.5743	0.0957	0.6785
Tabular	Extra Trees	0.9531	0.9779	0.6133	0.7356	−0.3397	0.6207	0.1003	0.6695
Tabular	MLP	0.9321	0.9549	0.6104	0.8104	−0.3217	0.6223	0.1047	0.8109
Tabular	Logistic Regression	0.8995	0.9365	0.5869	0.6638	−0.3127	0.5783	0.2217	0.6141

Table 9 Low-FPR operating points on the AndroZoo hold-out.

Rep.	Model	Thr.@1%	Rec.@1%	Thr.@5%	Rec.@5%
Seq.	BiLSTM	0.999	0.023	0.996	0.308
Seq.	Transformer	0.999	0.186	0.994	0.525
Tab.	XGBoost	0.995	0.074	0.989	0.184
Tab.	Random Forest	0.958	0.026	0.888	0.113
Tab.	Extra Trees	0.982	0.018	0.932	0.077
Tab.	MLP	–	0.000	–	0.000
Tab.	Logistic Reg.	–	0.000	1.000	0.026

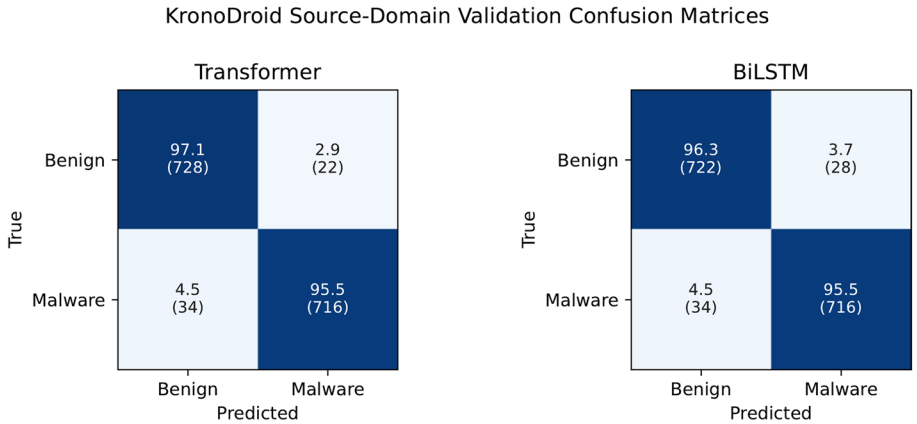


Fig. 8 KronoDroid validation confusion matrices for sequence probes.

scores (Table 9). This analysis is not used for model selection or calibration; it only reports what recall would remain if each source-trained probe were thresholded post hoc to satisfy FPR constraints of at most 1% or 5% on the temporal hold-out. The results show that the apparent fixed-threshold recall of tabular probes is not operationally stable: XGBoost recall falls to 7.4% at FPR $\leq$1% and 18.4% at FPR $\leq$5%. Sequence probes retain more signal in this conservative regime, especially the Transformer (18.6% and 52.5% recall at the same FPR limits), but even these values confirm that source-only temporal transfer is inadequate for deployment-grade low-FPR operation. Thus, the high-FPR results in Table 8 should be read as drift diagnostics rather than deployable operating points. A dash in Table 9 denotes that no nonzero-recall operating point satisfies the requested FPR constraint.

The confusion matrices reveal these asymmetric errors. Figures 8 and 9 show that the sequence probes are highly accurate on KronoDroid but separate into different temporal operating points on AndroZoo: the Transformer is more conservative and preserves benign specificity, while the BiLSTM recovers more recent malware. Tables 10 and 11 show a different pattern for aggregate behavioral profiles. Table 10 confirms that tabular models fit the historical source domain well, especially the tree ensembles. Table 11 then shows that this apparent source-domain separability does not transfer cleanly: most tabular models shift toward the malware class on recent benign applications, producing the elevated false-positive rates reported in Table 8. Thus, these confusion matrices are not redundant with the metric table; they identify whether temporal degradation appears as missed malware, benign over-flagging, or a trade-off between the two.

These results connect the detection experiment to the drift characterization above. The AndroZoo hold-out contains denser benign runtime traces, additional SDK- and service-related behavior tags, and category-level shifts that make historical behavioral cues less stable. Consequently, a model that appears highly reliable under KronoDroid validation can still suffer a large drop in future-domain macro-F1 and a large increase in error rates. The stronger temporal robustness of the sequence probes suggests that preserving event order helps retain discriminative context under drift, while aggregate tabular profiles are more vulnerable to benign-side behavioral modernization.

The feature-importance and drift-analysis results jointly explain the dominant failure mode. By Random Forest feature importance, the five most discriminative tabular features on KronoDroid capture session duration,

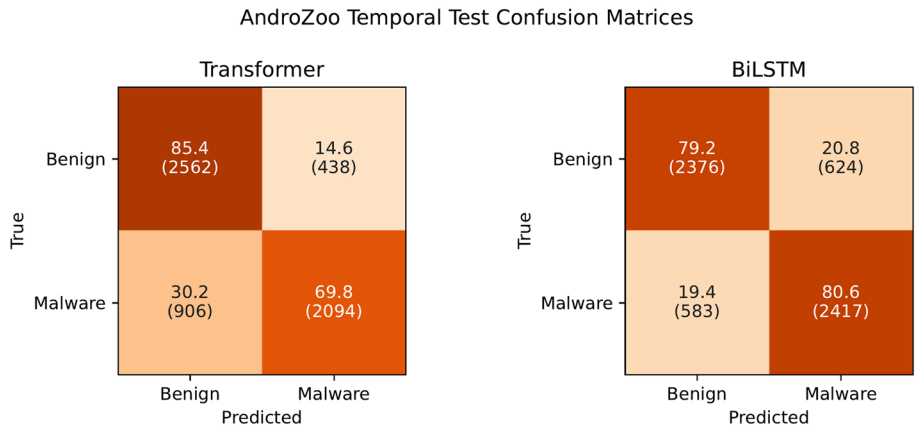


Fig. 9 AndroZoo temporal hold-out confusion matrices for sequence probes.

Table 10 Confusion matrices on the KronoDroid validation split for all five tabular probes (benign: 1,080; malware: 1,073). Rows: actual class; columns: predicted class (B = benign, M = malware).

		LR		RF		ET		MLP		XGBoost	
		$\hat{B}$	$\hat{M}$	$\hat{B}$	$\hat{M}$	$\hat{B}$	$\hat{M}$	$\hat{B}$	$\hat{M}$	$\hat{B}$	$\hat{M}$
Actual	**B**	1011	69	1060	20	1053	27	1034	46	1059	21
	M	147	926	78	995	74	999	100	973	77	996

Table 11 Confusion matrices on the AndroZoo temporal hold-out for all five tabular probes (benign: 3,000; malware: 3,000). Rows: actual class; columns: predicted class (B = benign, M = malware).

		LR		RF		ET		MLP		XGBoost	
		$\hat{B}$	$\hat{M}$	$\hat{B}$	$\hat{M}$	$\hat{B}$	$\hat{M}$	$\hat{B}$	$\hat{M}$	$\hat{B}$	$\hat{M}$
Actual	**B**	1265	1735	1277	1723	1138	1862	1133	1867	1679	1321
	M	665	2335	287	2713	301	2699	314	2686	434	2566

accessibility-query activity, burst intensity, device-ID access, and surveillance behavior. These signals were historically concentrated in malware traces. However, benign-side JSD analysis (Table 12) shows that the log-transformed file-read, file-write, and reflection features undergo the largest distributional shift between KronoDroid and AndroZoo benign distributions (JSD > 0.49). Modern benign applications increasingly trigger these behaviors through contemporary SDKs, authentication libraries, telemetry clients, and push-notification frameworks, causing the aggregate behavioral profile of recent benign apps to overlap substantially with historical malware signatures. This overlap is the direct mechanism behind the tabular FPR of 44.0% observed on AndroZoo: the model's most discriminative signals are exactly those that have become less class-specific over time. Addressing this failure therefore requires either future-domain adaptation data or drift-corrected feature representations, rather than post-hoc threshold adjustment alone.

Feature-group ablation: temporal stability by representation

To directly answer **RQ3**, we conduct a feature-group ablation using XGBoost as the probe model. Each condition either removes one of the four representation groups (leave-one-group-out) or trains on a single group exclusively (Table 13). The ablation reimplements the XGBoost pipeline with full-source-domain median imputation and no robust scaling to keep the ablation self-contained; accordingly, its full-model baseline (AZ F1 = 0.696) differs by <0.6 pp from the main experiment (Table 8, AZ F1 = 0.701), and all ablation comparisons should be read as internally consistent deltas rather than direct extensions of Table 8. In Table 13, ΔF1 is measured relative to the full ablation baseline; the dagger marks the temporal-feature removal condition that improves temporal F1, whereas the double dagger marks the sequence-only degenerate case discussed below.

Table 12 Top benign-side tabular shifts by Jensen–Shannon divergence.

Feature	Benign-side JSD
log1p_file_read_sensitive	0.585
log1p_file_write	0.582
log1p_reflection_invoke	0.517
early_file_write_flag	0.492
has_evasion_pattern	0.441
has_exfil_pattern	0.431
anti_to_total_ratio	0.396

Table 13 Feature-group ablation on the AndroZoo temporal hold-out.

Condition	#Feat	Krono F1	AZ F1	FPR	FNR	Δ F1
Baseline						
Full model (all groups)	165	0.954	0.696	0.422	0.178	—
Leave-one-group-out						
Remove Raw Counters	86	0.948	0.576	0.675	0.104	−0.119
Remove Derived Features	129	0.955	0.692	0.425	0.181	−0.003
Remove Temporal Features	143	0.950	**0.720**	**0.401**	0.150	+0.024
Remove Sequence Features	137	0.954	0.696	0.422	0.178	0.000
Single-group-only						
Only Raw Counters	79	0.942	**0.705**	0.427	0.153	+0.009
Only Derived Features	36	0.935	0.597	0.635	0.117	−0.099
Only Temporal Features	22	0.914	0.508	0.783	0.080	−0.188
Only Sequence Features	28	0.334	0.333	0.000	1.000	−0.362

[†]Removing temporal features improves temporal hold-out F1. [‡]The sequence-only setting is a degenerate classifier with FNR = 1.000.

Three findings emerge. First, *Raw Counters are the most temporally transferable among the evaluated tabular feature groups*: removing them collapses AndroZoo macro-F1 from 0.696 to 0.576 (−11.9 pp) and raises FPR to 67.5%, the largest degradation across any ablation condition. Conversely, using only Raw Counters (79 features) achieves macro-F1 0.705, which is *higher* than the full 165-feature model. This confirms that raw API call counts are the primary carrier of temporally stable discriminative signal.

Second, *Temporal Features (first-occurrence timestamps and inter-event deltas) are temporally fragile*: removing them *improves* AndroZoo macro-F1 by 2.4 pp (0.696→0.720) and reduces FPR from 42.2% to 40.1%. This counter-intuitive result indicates that absolute timing patterns learned from KronoDroid overfit to the historical behavioral cadence of the source domain; modern applications trigger the same APIs at different intervals, making timing-based features negative transfer signals under temporal evaluation.

Third, *sequence-chain tabular features are perfectly redundant*: removing them produces no change whatsoever (0.000 pp), and using them alone fails entirely (Krono macro-F1 0.334, FNR 100%), because ordered-chain flags are too sparse to train a useful XGBoost decision boundary without the supporting count context. This negative result indicates that the current chain design acts mainly as a sparse hand-engineered annotation layer rather than as an independent representation. We therefore do not interpret the sequence-chain tabular block as a useful contribution in its present form; future versions should either remove these flags from the tabular feature set or replace them with denser learned n-gram, motif, or sequence-embedding summaries that preserve order without collapsing into near-constant binary indicators.

Taken together, the ablation reveals that the temporal degradation of tabular probes is dominated by raw behavioral count drift—the benign-side SDK and service inflation that raises API-count profiles for recent benign apps toward historical malware baselines. Timing-based engineered features amplify this fragility rather than mitigating it, suggesting that future tabular representations for temporally robust Android malware detection should prioritize drift-stable raw behavioral counts over complex temporal derivations.

Exploratory drift-aware adaptation

The preceding sections constitute the primary contribution of the paper: measuring and explaining temporal behavioral drift under a strict historical-to-future protocol. This secondary analysis evaluates three lightweight adaptation strategies across all five tabular probes and both sequence models to examine how representation families respond when limited recent-domain evidence is available.

Drift-feature exclusion. Tabular models are retrained on KronoDroid after removing the top-K benign-side JSD features ($K \in \{10, 20, 30\}$), requiring no future-domain labels.

Conservative threshold recalibration. Source-trained models are evaluated at elevated decision thresholds ($t \in \{0.70, 0.80\}$) without parameter refitting. These sweeps are post-hoc operating-point diagnostics, not deployment-time calibration rules.

Few-shot supervised retrain or fine-tune. A stratified 5% or 10% subset of AndroZoo is used as labeled adaptation evidence. Tabular probes add this subset to KronoDroid and retrain from scratch; sequence probes fine-tune the KronoDroid checkpoint. For sequence probes, the protocol reserves an independent 10% AndroZoo validation subset for early stopping and leaves the remaining future-domain subset untouched for final testing. Consequently, the 5% and 10% adaptation conditions use final-test subsets of different sizes after the fixed validation carve-out, so non-monotonic differences should be interpreted jointly as adaptation-size and split-sensitivity effects rather than as a purely sample-size-driven trend. We repeat this fine-tuning protocol over six random seeds (11, 22, 33, 42, 44, and 55) and report mean ± standard deviation. These seeds affect only the AndroZoo adaptation/validation/final-test re-partitions and fine-tuning stochasticity; the KronoDroid source checkpoints are fixed and are not retrained over multiple seeds.

Upper-bound estimates. Tabular upper bounds are 5-fold cross-validation results on the balanced AndroZoo hold-out; sequence upper bounds use 80% AndroZoo fine-tuning, 10% validation, and 10% untouched final testing, repeated over the same six seeds. Tables 14, 15, and 16 report macro-F1 with FPR in parentheses where applicable.

Tabular probe results (Table 14). XGBoost is the strongest tabular baseline (macro-F1 0.701, FPR 44.0%) and responds most aggressively to supervised adaptation: 300 labeled future-domain samples raise macro-F1 to 0.847 and reduce FPR to 15.5%, while 600 samples raise macro-F1 to 0.878 and reduce FPR to 10.3%, closing approximately 80% of the gap to the 5-fold CV upper bound (0.922). Drift-feature exclusion does not consistently help XGBoost because the most-drifted features remain discriminative; thresholding at $t = 0.70$ lowers FPR to 36.8% with little macro-F1 change but higher FNR.

RandomForest and ExtraTrees show the steepest few-shot recovery: at 5% adaptation, RF gains +17.3 macro-F1 pp and ET gains +19.4 pp, with FPR falling from 57.4%/62.1% to 24.9%/27.2%. Threshold recalibration at $t = 0.70$ is also useful for these ensembles (+7.4 pp for RF, +10.4 pp for ET). Drift-feature exclusion is counterproductive for RF but helps ET at $K = 20$ (+5.1 pp, FPR 53.1%), suggesting different dependence on drifted features.

Logistic Regression benefits monotonically from drift-feature exclusion ($K = 10$: +9.8 pp; $K = 20$: +10.2 pp; $K = 30$: +11.9 pp), consistent with a linear scorer whose additive weights are sensitive to benign-side feature shift. Few-shot retraining further improves LR to macro-F1 0.724 at 10%, although its upper bound remains the lowest (0.870). MLP behaves less regularly: $K = 10$ hurts, $K = 20$ gives the largest drift-exclusion gain (+7.3 pp), and meaningful supervised recovery appears only at 10% adaptation (macro-F1 0.779, FPR 29.9%).

Cross-model comparison (Table 15). Few-shot retraining benefits all tabular probes, but tree ensembles recover fastest. At 10% adaptation, XGBoost achieves the highest F1 (0.878) and lowest FPR (10.3%), followed by RF (0.843, 18.9%) and ET (0.833, 20.8%), while MLP (0.779) and LR (0.724) trail. Similar full-hold-out upper bounds (0.870–0.922) indicate that the temporal gap is driven more by distribution shift and sample efficiency than by model capacity alone.

Sequence models: temporal resilience with protocol-clean fine-tuning. BiLSTM and Transformer have stronger unadapted temporal baselines than tabular probes (macro-F1 0.799 and 0.775; FPR 20.8% and 14.6%).

Table 14 Adaptation results for tabular probes on the AndroZoo hold-out.

Strategy	LR	RF	ET	MLP	XGBoost
No adaptation (baseline)	0.587 (0.578)	0.645 (0.574)	0.613 (0.621)	0.610 (0.622)	0.701 (0.440)
Label-free post-hoc strategies					
Threshold recalib. (t = 0.70)	0.617 (0.483)	0.719 (0.378)	0.717 (0.407)	0.633 (0.565)	0.701 (0.368)
Threshold recalib. (t = 0.80)	0.627 (0.439)	0.656 (0.221)	0.720 (0.274)	0.644 (0.533)	0.686 (0.329)
Drift-feature excl. (K = 10)	0.685 (0.394)	0.639 (0.580)	0.615 (0.616)	0.592 (0.653)	0.669 (0.439)
Drift-feature excl. (K = 20)	0.689 (0.320)	0.623 (0.607)	0.665 (0.531)	0.683 (0.495)	0.642 (0.471)
Drift-feature excl. (K = 30)	0.706 (0.334)	0.626 (0.601)	0.629 (0.603)	0.643 (0.578)	0.684 (0.460)
Supervised few-shot retrain					
5% (n = 300)	0.704 (0.400)	0.817 (0.249)	0.808 (0.272)	0.617 (0.615)	0.847 (0.155)
10% (n = 600)	0.724 (0.359)	0.843 (0.189)	0.833 (0.208)	0.779 (0.299)	0.878 (0.103)
Full retrain (upper bound, 5-fold CV)	0.870 (—)	0.920 (—)	0.913 (—)	0.885 (—)	0.922 (—)

Table 15 Cross-model adaptation summary on the AndroZoo hold-out.

[†]Threshold column reports the t = 0.70 conservative threshold setting for each model. [‡]Few-shot column reports the 10% supervised adaptation setting.

Family	Model	Baseline F1	Thresh. F1 (FPR)	Few-Shot 10% F1 (FPR)	Upper Bound F1
Tabular	LR	0.587	0.617 (0.483)	0.724 (0.359)	0.870
	RF	0.645	0.719 (0.378)	0.843 (0.189)	0.920
	ET	0.613	0.717 (0.407)	0.833 (0.208)	0.913
	MLP	0.610	0.633 (0.565)	0.779 (0.299)	0.885
	XGB	0.701	0.701 (0.368)	0.878 (0.103)	0.922
Sequence	BiLSTM	0.799	0.798 (0.179)	0.823±0.024 (0.149)	0.909±0.010
	Transf.	0.775	0.772 (0.130)	0.818±0.004 (0.182)	0.904±0.006

Table 16 Multi-seed sequence adaptation results on the AndroZoo hold-out using independent validation and untouched final-test splits. Values report mean ± standard deviation over six AndroZoo re-partitions and fine-tuning runs; the KronoDroid source checkpoints are fixed.

Model	Strategy	Macro-F1	ROC-AUC	FPR	FNR
BiLSTM	No adaptation	0.799 ± 0.000	0.837 ± 0.000	0.208 ± 0.000	0.195 ± 0.000
	Threshold t = 0.70	0.798 ± 0.000	0.837 ± 0.000	0.179 ± 0.000	0.225 ± 0.000
	Threshold t = 0.80	0.792 ± 0.000	0.837 ± 0.000	0.160 ± 0.000	0.254 ± 0.000
	Few-shot 5% (n = 300)	0.835 ± 0.009	0.888 ± 0.005	0.136 ± 0.023	0.193 ± 0.009
	Few-shot 10% (n = 600)	0.823 ± 0.024	0.881 ± 0.015	0.149 ± 0.050	0.203 ± 0.008
	Full fine-tune 80/10/10	0.909 ± 0.010	0.954 ± 0.008	0.052 ± 0.022	0.129 ± 0.026
Transformer	No adaptation	0.775 ± 0.000	0.852 ± 0.000	0.146 ± 0.000	0.302 ± 0.000
	Threshold t = 0.70	0.772 ± 0.000	0.852 ± 0.000	0.130 ± 0.000	0.321 ± 0.000
	Threshold t = 0.80	0.769 ± 0.000	0.852 ± 0.000	0.123 ± 0.000	0.334 ± 0.000
	Few-shot 5% (n = 300)	0.804 ± 0.006	0.870 ± 0.003	0.198 ± 0.018	0.194 ± 0.013
	Few-shot 10% (n = 600)	0.818 ± 0.004	0.880 ± 0.003	0.182 ± 0.020	0.182 ± 0.014
	Full fine-tune 80/10/10	0.904 ± 0.006	0.954 ± 0.006	0.052 ± 0.017	0.139 ± 0.019

Thresholding at t = 0.70 reduces FPR to 17.9% and 13.0% with little macro-F1 change, showing that event-order representations better preserve context under benign-side drift.

Under the independent-validation protocol (Table 16), few-shot sequence fine-tuning improves both sequence probes rather than collapsing under a single split. BiLSTM reaches 0.835±0.009 macro-F1 with 5% adaptation and 0.823±0.024 with 10% adaptation, while Transformer reaches 0.804±0.006 and 0.818±0.004, respectively. Thus, sequence adaptation is beneficial but not strictly monotonic: BiLSTM shows larger seed/split sensitivity at 10%, whereas Transformer improves more steadily. Full sequence fine-tuning provides similar upper-bound capacity for both probes (BiLSTM 0.909±0.010, Transformer 0.904±0.006), confirming that target-domain evidence can recover much of the temporal degradation when validation and final testing are separated.

Synthesis: complementary adaptation profiles. The results reveal two deployment profiles. Sequence models are more robust without adaptation (+9.8 macro-F1 pp and −23.2 FPR pp over the best tabular baseline), whereas tabular tree ensembles are highly sample-efficient once a small labeled future-domain set is available (XGBoost gains +17.7 macro-F1 pp at 10% adaptation). The sequence fine-tuning results narrow this contrast: neural sequence probes also benefit from limited target labels, but their gains are smaller than XGBoost's and BiLSTM remains more seed-sensitive than Transformer. Thus, adaptation efficiency is empirically separable from baseline temporal resilience and should guide long-term detector selection.

Threats to validity

Several limitations should be considered when interpreting the results. First, dynamic analysis observes only behavior triggered during the controlled 120-second execution window. Dormant, trigger-dependent, or environment-aware logic may remain unobserved, and we do not measure instruction- or method-level coverage. The parallel emulator protocol, fallback launch strategy, deterministic stimulation, Monkey interaction, and timeout handling reduce collection instability, but future work should quantify whether stimulation effectiveness differs across domains. The runtime-activity proxy statistics in Table 6 show that recent AndroZoo samples generate dense monitored traces, but they cannot prove that comparable fractions of application code or malicious payload logic were executed.

Second, all APKs are analyzed on a controlled Android 8.0 emulator. This stable longitudinal baseline supports Frida hooks, emulator reproducibility, and broad compatibility across the 2008–2026 timeline, but it may underrepresent behaviors tied to newer Android APIs or device configurations. A newer OS image could expose more modern API paths for recent applications while introducing backward-compatibility failures for older samples; some measured drift may therefore reflect interaction between ecosystem evolution and the fixed analysis environment. Accordingly, the reported drift should be read as drift observed under a fixed Android 8.0 dynamic-analysis environment. A multi-OS rerun using newer Android images would be required to quantify how much of the measured shift is caused by platform incompatibility rather than behavioral concept drift alone.

Third, VirusTotal consensus is an operational proxy rather than manual ground truth. We reduce label noise by excluding ambiguous low-positive samples and using conservative criteria (at least three positive engines for malware and zero positives for benign applications), but sensitivity analysis over stricter thresholds such as VT$\geq$ 5 remains necessary. The AndroZoo temporal hold-out is also balanced by design to isolate temporal generalization and representation drift, not to estimate real-world prevalence; precision under low-prevalence deployment should be evaluated separately.

Fourth, the primary source-training experiments use a fixed seed for deterministic reproduction, whereas the sequence-adaptation analysis reports six-seed estimates with independent validation and untouched final testing. The six-seed estimates reduce the risk that the BiLSTM/Transformer fine-tuning conclusion is an artifact of one target split. The remaining limitation is that tabular adaptation is still reported from the original deterministic split and the upper-bound protocols differ across representation families: tabular upper bounds use five-fold AndroZoo cross-validation, whereas sequence upper bounds use repeated 80/10/10 fine-tuning splits. In addition, the sequence 5% and 10% few-shot settings leave different final-test sizes after the fixed validation subset, so their comparison captures both adaptation-data effects and split sensitivity. Therefore, the adaptation study should still be interpreted as an operational sensitivity analysis rather than a complete hyperparameter-optimized continual-learning benchmark.

Fifth, verified family labels are unavailable for all target samples, so the results should be interpreted as temporal behavioral transfer rather than family-stratified attribution. The evaluated models are representative probes rather than exhaustively optimized detectors, which is intentional because the goal is to compare representation transfer under the same source-only protocol. Sequence traces are capped at 256 events; inspection shows this affects only 1.5% of AndroZoo benign traces and 0.4% of malware traces, but open-vocabulary encoders, longer

contexts, or semantic tag normalization may recover additional future-domain signal. Finally, the study uses one historical source domain and one recent temporal hold-out; larger longitudinal slices, family-aware stratification, alternative VirusTotal thresholds, additional seeds, and additional temporal datasets would clarify whether the observed drift patterns are monotonic, family-specific, or concentrated in particular application categories. Because the behavior-category labels are produced consistently by the instrumentation taxonomy, the category-level drift results are more reliable than an inferred malware-family analysis would be for the present data.

Conclusion

This paper presented DriftDroid, a temporal behavioral drift study for Android malware detection. Rather than proposing a new detector architecture, the work evaluates how historical runtime-behavior evidence transfers to recent Android applications. KronoDroid is used as the historical source domain, while a 2024–2026 AndroZoo hold-out selected with explicit VirusTotal criteria provides the future-domain evaluation setting. This design separates source-domain learning from temporal generalization and avoids using future samples during preprocessing, model selection, or thresholding.

The empirical results show that temporal drift is both measurable and operationally significant. Source-domain validation produces strong results for both sequence and tabular probes, but all models degrade on the future-domain hold-out. The degradation is representation dependent. Sequence models retain the strongest temporal performance, with the BiLSTM reaching 0.7988 macro-F1 on AndroZoo and the Transformer retaining the highest AndroZoo ROC-AUC among the evaluated probes. Tabular models, despite strong KronoDroid validation scores, degrade more sharply; their dominant error mode is a high false-positive rate on recent benign applications. This indicates that aggregate behavioral indicators learned from historical data can become unreliable as benign applications increasingly exhibit network, file, cryptographic, SDK, and background-service behaviors that previously looked suspicious.

As a secondary operational analysis, the adaptation experiments show that drift is not easily remedied by post-hoc adjustments and that the two model families respond differently when limited recent-domain evidence becomes available. Applied across all five tabular probes, drift-feature exclusion is generally ineffective for tree models (the most-drifted features are simultaneously the most discriminative), while conservative threshold recalibration is effective for RandomForest and ExtraTrees (+7.4 and +10.4 macro-F1 pp respectively in post-hoc threshold sweeps). Few-shot supervised retraining consistently bridges the gap for all tabular models at the 10% adaptation setting: macro-F1 increases by 14–22 pp across architectures, closing 48–80% of the performance gap to the full-retrain upper bound (72–80% for tree-ensemble probes; MLP and LR recover more modestly at 62% and 48% respectively, consistent with the greater inertia of neural and linear representations under small-set retraining). Sequence models present the complementary profile: their event-order representations are inherently more robust to benign-side drift (+9.8 macro-F1 pp and −23.2 FPR pp over the best tabular baseline without any adaptation), and the six-seed fine-tuning protocol improves mean macro-F1 over the unadapted baselines for both sequence probes. BiLSTM improves to 0.835±0.009 macro-F1 with 5% adaptation and 0.823±0.024 with 10%, while Transformer improves to 0.804±0.006 and 0.818±0.004. These results no longer support a simple catastrophic-forgetting interpretation; instead, they indicate beneficial but seed-sensitive neural adaptation, especially for BiLSTM. The primary evidence therefore remains the clean unadapted temporal evaluation and the behavioral drift analysis: robust Android malware evaluation requires explicit temporal hold-outs and representation-aware drift analysis, while long-term detector reliability will likely require mechanisms to incorporate recent-domain evidence.

Future work should explore several directions to extend these findings. First, *drift-aware feature regularization* during source-domain training—penalizing reliance on features with high benign-side JSD—could improve temporal robustness without requiring future-domain labels. Second, *temporally invariant behavioral representations* that explicitly separate stable malware-indicative patterns from evolving benign SDK behaviors may reduce the

aggregate feature aging observed here; a direct TIF-style or invariant-risk baseline on the same dynamic features would be a natural next comparison. Third, *semantic normalization for future-domain sequence tags* should be evaluated to determine whether open-vocabulary or API-ontology encoders can retain the strict temporal protocol while reducing the information loss caused by UNK mapping. Fourth, *rehearsal-based continual fine-tuning* and regularized neural updating should be evaluated for sequence models to determine whether target-domain gains can be improved while preserving source-domain behavior. Fifth, *longitudinal drift analysis* across multiple annual slices, Android platform versions, and VirusTotal threshold settings would clarify whether the behavioral shifts observed here are monotonic, accelerating, environment-sensitive, or concentrated around specific Android API transitions.

Author contributions Conceptualization, Muhammet Tan and Halit Bakır; Methodology, Muhammet Tan and Halit Bakır; Software, Muhammet Tan; Validation, Muhammet Tan, Halit Bakır, Eid Albalawi, and A. H. Abdul Hafez; Formal analysis, Muhammet Tan and Halit Bakır; Investigation, Muhammet Tan; Data curation, Muhammet Tan; Writing—original draft preparation, Muhammet Tan; Writing—review and editing, Halit Bakır, Eid Albalawi, and A. H. Abdul Hafez; Visualization, Muhammet Tan; Supervision, Halit Bakır and A. H. Abdul Hafez; Project administration, Halit Bakır. All authors have read and agreed to the published version of the manuscript.

Funding This work was supported by the Deanship of Scientific Research, Vice Presidency for Graduate Studies and Scientific Research, King Faisal University, Saudi Arabia [Grant No: KFU265184].

Data availability The study-specific dynamic-analysis artifacts, feature schemas, tabular feature files, event-sequence JSONL files, experiment notebooks, model-training scripts, inference scripts, and aggregate outputs are publicly available at https://github.com/muhammettan28/kangalguard-dynamic and archived on Zenodo at https://doi.org/10.5281/zenodo.22696595. The repository includes the AndroZoo tabular and sequence-log files under the `androzoo/` directory and the corresponding KronoDroid-derived artifacts together with the experiment code used to reproduce the tabular, sequence, ablation, adaptation, and visualization analyses. APK binaries from KronoDroid and AndroZoo are not redistributed; users must obtain original APKs through the corresponding dataset access channels.

Declarations

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

1. Kaspersky Lab. Mobile malware evolution 2023. *Kaspersky SecureList Report*. Available: https://securelist.com/mobile-malware-evolution-2023/111681/ (2023).
2. Kaspersky Lab. Banking data theft attacks on smartphones triple in 2024, kaspersky reports. Kaspersky Press Release (2025). Accessed: May 17, 2026. Available: kaspersky.com press release.
3. Shatnawi, A. S., Yassen, Q. & Yateem, A. An android malware detection approach based on static feature analysis using machine learning algorithms. *Procedia Comput. Sci.* **201**, 653–658 (2022).
4. Pathak, A., Kumar, T. S. & Barman, U. Static analysis framework for permission-based dataset generation and android malware detection using machine learning. *EURASIP Journal on Information Security*. **2024**, 33. https://link.springer.com/article/10.1186/s13635-024-00182-3 (2024).

5. Pang, J. & Bian, J. Android malware detection based on naive bayes. *Proceedings of the IEEE International Conference on Software Engineering and Service Sciences, ICSESS*, 483–486 https://ieeexplore.ieee.org/document/9040796 (2019).
6. Sathyadevi, G., Abishek, J. & Shakthieiswaran, B. S. Dynashield:android malware detection using dynamic analysis of network traffic. *2024 International Conference on System, Computation, Automation and Networking, ICSCAN 2024* https://ieeexplore.ieee.org/document/10894660 (2024).
7. Ciaramella, G., Mercaldo, F. & Santone, A. Dynamic analysis for explainable fine-grained android malware detection. *Lect. Notes Comput. Sci.* **15235**, 110–127. https://doi.org/10.1007/978-3-031-76371-7_8 (2025).
8. Enriquez, E. G. V. & Gutiérrez-Cárdenas, J. Dynamic malware analysis using machine learning-based detection algorithms. *Interfases* https://doi.org/10.26439/interfases2024.n19.7097 (2024).
9. Singh, A., Tanha, M., Girdhar, Y. & Hunter, A. Interpretable android malware detection based on dynamic analysis. In *International Conference on Information Systems Security and Privacy*, vol. 1, 195–202 (Science and Technology Publications, Lda, 2024).
10. Rashid, M. U. et al. Hybrid android malware detection and classification using deep neural networks. *Int. J. Comput. Intell. Syst.* **18**, 52 https://link.springer.com/article/10.1007/s44196-025-00783-x (2025).
11. Kılıç, K. İsmail Atacak & İbrahim Alper Doğru. Fabldroid: Malware detection based on hybrid analysis with factor analysis and broad learning methods for android applications. *Eng. Sci. Technol. Int. J.* **62**, 101945. https://doi.org/10.1016/j.jestch.2024.101945 (2025).
12. Arora, A. H-pintdroid: Hybrid android malware detection using permissions, intents and network traffic. In *Conference Proceedings - 2025 4th International Conference on Advances in Computing, Communication, Embedded and Secure Systems, ACCESS 2025*, 481–486 (Institute of Electrical and Electronics Engineers Inc., 2025).
13. Allix, K. et al. Empirical assessment of machine learning-based malware detectors for android. *Empir. Softw. Eng.* **21**, 183–211 (2016).
14. Pendlebury, F., Pierazzi, F., Jordaney, R., Kinder, J. & Cavallaro, L. TESSERACT: Eliminating experimental bias in malware classification across space and time. In *Proceedings of the 28th USENIX Security Symposium*, 729–746 (USENIX Association, 2019).
15. Kan, Z. et al. TESSERACT: Eliminating experimental bias in malware classification across space and time (extended version). *CoRR* arXiv:2402.01359 (2025).
16. Guerra-Manzanares, A., Bahsi, H. & Nõmm, S. Kronodroid: Time-based hybrid-featured dataset for effective android malware detection and characterization. *Comput. Secur.* **110** (2021).
17. Guerra-Manzanares, A. Machine learning for android malware detection: Mission accomplished? A comprehensive review of open challenges and future perspectives. *Comput. Secur.* **138**, 103654 (2024).
18. Li, L. et al. Androzoo++: Collecting millions of android apps and their metadata for the research community. *arXiv preprint* arXiv:1709.05281 (2017).
19. Saufan, M. H. M. et al. Random forest-based approach to android malware detection in machine learning. *2024 5th International Conference on Artificial Intelligence and Data Sciences, AiDAS 2024 - Proceedings* 381–385 https://ieeexplore.ieee.org/document/10730240 (2024).
20. Misalkar, H. D. & Harshavardhanan, P. Tdbamla: Temporal and dynamic behavior analysis in android malware using LSTM and attention mechanisms. *Comput. Stand. Interfaces* https://doi.org/10.1016/j.csi.2024.103920 (2025).
21. Youssef, N., Elbaraway, N. & Elmaghraby, A. Transformer-based api call sequence modeling for dynamic malware detection. In *Conference Proceedings - IEEE SOUTHEASTCON*, 494–500 (Institute of Electrical and Electronics Engineers Inc., 2025).
22. Youssef, N., Elbarawi, N. & Elmaghraby, A. Bert for dynamic malware detection: a sequence-based classification approach. In *Proceedings of the IEEE Symposium on Computers and Communications*, 1–6 (Institute of Electrical and Electronics Engineers IEEE, 2026).
23. Qian, L. & Cong, L. Channel features and API frequency-based transformer model for malware identification. *Sensors* https://doi.org/10.3390/s24020580 (2024).
24. Trizna, D., Demetrio, L., Biggio, B. & Roli, F. Nebula: Self-attention for dynamic malware analysis. *IEEE Trans. Inf. Forensics Secur.* **19**, 6155–6167. https://doi.org/10.1109/TIFS.2024.3409083 (2024).
25. Singh, P., Borgohain, S. K. & Kumar, J. Investigation and pre-processing of clamp mlaware dataset for machine learning models. In *2022 6th International Conference on Electronics, Communication and Aerospace Technology*, 481–486 (IEEE, 2022).
26. Singh, P., Borgohain, S. K. & Kumar, J. Performance enhancement of svm-based ml malware detection model using data preprocessing. In *2022 2nd International Conference on Emerging Frontiers in Electrical and Electronic Technologies (ICEFEET)*, 1–4 (IEEE, 2022).
27. Singh, P., Borgohain, S. K., Sharma, L. D. & Kumar, J. Minimized feature overhead malware detection machine learning model employing mrmr-based ranking. *Concurrency and Computation: Practice and Experience* **34**, e6992 (2022).
28. Singh, P., Borgohain, S. K. & Kumar, J. Optimized feature representation and selection for malware detection using portable executable headers and machine learning. *J. Comput. Virol. Hacking Tech.* **21**, 10 (2025).

29. Singh, P., Borgohain, S. K., Sarkar, A. K., Kumar, J. & Sharma, L. D. Feed-forward deep neural network (ffdnn)-based deep features for static malware detection. *Int. J. Intell. Syst.* **2023**, 9544481 (2023).
30. Mohawesh, R. et al. Cybersecurity challenges and solutions in 5g-enabled internet of things. *Discov. Comput.* **29**, 29 (2026).
31. Mohawesh, R., Maqsood, S., Jararweh, Y. & Bany Salameh, H. Federated learning support for cybersecurity: Fundamentals, applications, and opportunities. In *2023 International Conference on Intelligent Computing, Communication, Networking and Services (ICCNS)*, 50–56 (IEEE, 2023).
32. Mohawesh, R., Ottom, M. A. & Bany Salameh, H. A data-driven risk assessment of cybersecurity challenges posed by generative ai. *Decis. Anal. J.* **15**, 100580 (2025).
33. Fu, A., Pei, H. & Tanha, M. Self-supervised learning for android malware detection on a time-stamped dataset. *CoRR* arXiv: abs/2604.23025 (2026). https://arxiv.org/abs/2604.23025.
34. Narayanan, A., Yang, L., Chen, L. & Liu, J. Adaptive and scalable android malware detection through online learning. In *Proceedings of the International Joint Conference on Neural Networks*, 2484–2491 (2016).
35. Narayanan, A., Chandramohan, M., Chen, L. & Liu, Y. Context-aware, adaptive, and scalable android malware detection through online learning. *IEEE Trans. Emerg. Top. Comput. Intell.* **1**, 157–175 (2017).
36. Mirzaei, O., de Fuentes, J. M., Tapiador, J. M. E. & González-Manzano, L. ANDRODET: An adaptive android obfuscation detector. *Future Gener. Comput. Syst.* **90**, 240–261 (2019).
37. Xu, K., Li, Y., Deng, R. H., Chen, K. & Xu, J. DroidEvolver: Self-evolving android malware detection system. In *Proceedings of the 4th IEEE European Symposium on Security and Privacy*, 47–62 (IEEE, 2019).
38. Jordaney, R. Transcend: Detecting concept drift in malware classification models. In *Proceedings of the 26th USENIX Security Symposium* 625–642 (USENIX Association, 2017).
39. Barbero, F., Pendlebury, F., Pierazzi, F. & Cavallaro, L. Transcending TRANSCEND: Revisiting malware classification in the presence of concept drift. In *Proceedings of the 2022 IEEE Symposium on Security and Privacy*, 805–823 (IEEE, 2022).
40. Guerra-Manzanares, A. & Bahsi, H. Experts still needed: Boosting long-term android malware detection with active learning. *J. Comput. Virol. Hacking Tech.* **20**, 901–918 (2024).
41. Muzaffar, A., Hassen, H. R., Zantout, H. & Lones, M. A. ActDroid: An active learning framework for android malware detection. *Comput. Secur.* **160** (2026).
42. Zhang, X. Enhancing state-of-the-art classifiers with api semantics to detect evolved android malware. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security* 757–770 (ACM, 2020).
43. Zheng, X., Yang, S., Ngai, E. C. H., Jana, S. & Cavallaro, L. Learning temporal invariance in android malware detectors. *CoRR* arXiv: abs/2502.05098 (2025).
44. Najibi, M. & Bidgoly, A. J. Towards a robust android malware detection model using explainable deep learning. *J. Inf. Secur. Appl.* **93**, 104191 (2025).
45. Zola, F., Bruse, J. L. & Galar, M. Temporal analysis of distribution shifts in malware classification for digital forensics. In *Proceedings of the 8th IEEE European Symposium on Security and Privacy Workshops*, 439–450 (IEEE, 2023).
46. Quertier, T. A lean transformer model for dynamic malware analysis and detection. arXiv preprint. arXiv:2408.02313 (2024).
47. Anusree K. J., Patel, N. D., Saravanan, D. & Patel, A. Comparative study of natural language processing models for malware detection using api call sequences. https://www.preprints.org/manuscript/202602.1359/v1 (2026).

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Muhammet Tan[1] · Halit Bakır[1] · Eid Albalawi[2] · A. H. Abdul Hafez[2]

✉ A. H. Abdul Hafez
aabdulhafiz@kfu.edu.sa

[1] Department of Computer Engineering, Sivas University of Science and Technology, Sivas, Turkey

[2] Department of Computer Science, College of Computer Sciences and Information Technology, Al-Ahsa, Saudi Arabia